Orbital Sentinel

Early-warning health monitoring for satellite fleets:
telemetry anomaly detection, explanation and risk ranking on a live 3D globe

Software Engineering Lab (BCSE301P)
Experiment 10: Project Report

Submitted by
Pushkar Kumar Mishra
Register No. 24BPS1128
B.Tech CSE (Cyber Physical Systems)
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai

22 October 2026

Abstract

A satellite reports thousands of telemetry channels, far more than an operator can watch by hand. Orbital Sentinel learns the normal behaviour of each channel with a small LSTM network, replays NASA's expert-labelled SMAP and Curiosity (MSL) telemetry through those models, and flags readings that the model did not expect. Each anomaly is attributed to the channel that caused it, held back by a persistence filter until it has lasted long enough to matter, given a severity and a 1 to 10 risk level, and ranked across a fleet of five satellites. Analysts work from a risk-ordered alert list, a health panel with the telemetry chart, and a 3D globe that shows the five satellites at their real current positions, computed in the browser from public TLE data.

Measured against NASA's labels on all 81 labelled channels, the detector reaches precision 0.683, recall 0.683 and F1 0.683, with 2.24% of normal points flagged. On held-out data the persistence filter (N = 2) lowers the share of falsely flagged points from 3.23% to 2.83% at the cost of three missed anomalies. The system was built with an incremental process model with a prototype spike, tracked in Jira for the final increment, and tested with 219 pytest tests, a 15-case Selenium IDE suite and JMeter load, stress, spike and soak scenarios. It is a proof of concept: health telemetry is a replay of historical data and only the orbital positions are live.

Contents

Abstract 2

Contents 2

1. Introduction 4

   1.1 Background and motivation 4

   1.2 Problem statement 4

   1.3 Objectives 4

   1.4 Scope and limitations 4

2. Literature Survey and Existing Systems 6

   2.1 Review of existing systems 6

   2.2 Gaps identified and proposed solution 6

3. Software Process Model and Project Planning 7

   3.1 SDLC model chosen and justification 7

   3.2 Project schedule 7

   3.3 Risk analysis 8

   3.4 Effort estimation (basic COCOMO, organic mode) 8

4. Requirements Analysis 9

   4.1 Functional requirements 9

   4.2 Non-functional requirements 9

   4.3 SRS summary 10

   4.4 Hardware and software requirements 10

5. System Design 11

   5.1 System architecture 11

5.2 Use case diagram 12
5.3 Class diagram 13
5.4 Behavioural and interaction diagrams 15
5.5 Database design 16
5.6 UI design 16
6. Implementation 18
6.1 Technology stack 18
6.2 Module description 18
6.3 Key code snippets 19
7. Software Tools Used 21
7.1 Jira Software (planning and tracking) 21
7.2 Git and GitHub with GitHub Actions (version control and CI) 24
7.3 StarUML (UML modelling) 27
7.4 Selenium IDE (functional UI testing) 28
7.5 Apache JMeter (load and performance testing) 29
7.6 Summary of tools 30
8. Testing 32
8.1 Test plan and strategy 32
8.2 Test cases 32
8.3 Test results and bug log 35
9. Results and Discussion 37
9.1 Output screenshots of the working system 37
9.2 Analysis against the objectives 39
10. Conclusion and Future Enhancements 41
11. References 42
Appendix A: Project Links 43
Appendix B: How to run the project 44

1. Introduction

1.1 Background and motivation

Navigation, weather forecasting, disaster response, communications and Earth observation all depend on satellites run by agencies such as ISRO, NASA and ESA. When a satellite starts to fail, the services built on it degrade, and the spacecraft itself is usually worth hundreds of millions of rupees or dollars. Failures rarely arrive without warning: a power channel sags, a component runs hot, or a sensor drifts for some time before anything breaks.

The difficulty is volume. One spacecraft reports thousands of telemetry channels. Fixed out-of-limit alarms catch values that cross a hard line, but they miss values that are within limits and still wrong for the context, and they raise many alarms for harmless spikes. Operators need help to see which of many small irregularities deserve attention first.

Operational telemetry from real missions is not public. This project therefore uses the expert-labelled telemetry that NASA released for the SMAP satellite and the Curiosity (MSL) rover together with Hundman et al. [1]. The method itself is general and would apply to any system with many sensor channels, such as power grids, industrial machines or hospital monitors.

1.2 Problem statement

Given a stream of multichannel spacecraft telemetry, detect anomalous behaviour as it unfolds, explain which channel is responsible, suppress transient false alarms, estimate how serious each anomaly is, and rank the satellites in a fleet by risk, so that an analyst can see at a glance where to look and can drill down to the evidence. The system must report its own accuracy against ground-truth labels, and it must show where each satellite is, so that health and position can be read together.

1.3 Objectives

1. Detect anomalies in spacecraft telemetry with a learned model of normal behaviour, and measure accuracy against ground-truth labels (REQ-1 to REQ-6).
2. Explain each anomaly by naming the responsible channel, suppress transient false alarms, assign a severity and risk level, and rank satellites by risk (REQ-7 to REQ-11). This explanation and prioritisation layer is the part this project adds on top of the published detector.
3. Track real satellites live on a 3D globe, coloured by risk, with click-to-inspect (REQ-12 to REQ-15).
4. Give analysts a risk-ranked alert list with drill-down to the telemetry, and log every detection (REQ-16 to REQ-18).
5. Enforce three roles (Analyst, Administrator, Evaluator) and meet the non-functional targets in the SRS.

1.4 Scope and limitations

In scope is everything in the SRS, demonstrated on public data on a single laptop. Out of scope are integration with an operator's private live telemetry, failure forecasting, multi-user deployment and GPU training.

Three statements frame every result in this report:

- The system detects anomalies as they happen. It does not forecast or predict failures.
- Health telemetry is a historical replay of a public NASA dataset, not a live feed. Only the orbital positions are live.

- It is a proof of concept for decision support, not an operational monitoring tool. Four of the five satellites are proxies: real satellites at their real positions whose health comes from replayed SMAP or MSL channels. The interface says so on every proxy.

Known limitations: detection quality is moderate (F1 0.683 overall, weaker on MSL than on SMAP); the dynamic threshold's absolute guard means channels with tiny absolute errors, such as A-1, are never flagged; and a replay lasts only 28 to 105 windows per satellite, after which restarting it reproduces the same alerts. The backend is sized for a small team: above about 55 users logging in at the same moment, requests time out (O-03).

2. Literature Survey and Existing Systems

2.1 Review of existing systems

Mission control software.

Most ground systems still rely on out-of-limits checking: each channel has fixed red and yellow limits and an alarm fires when a value crosses one. This is simple and well understood, but it cannot see a value that is in range and still abnormal for the current mode, and it gives no ranking when several channels misbehave at once. NASA's open-source Open MCT [14] is a good example of modern mission-control visualisation; it plots telemetry and shows limit states, but it does not learn what normal looks like.

Learned detectors from research.

Hundman et al. [1] train one LSTM per channel on normal data, predict the next values, and flag points whose smoothed prediction error exceeds a non-parametric dynamic threshold. Their code (Telemanom) and the SMAP/MSL dataset are public, and this project follows their method. OmniAnomaly [15] models all channels jointly with a stochastic recurrent network and reports strong results on the same data. Simpler baselines include Isolation Forest [4] and autoencoders. These papers report accuracy, but they stop at a score; none of them ships an operator interface, an explanation per alert, or a way to rank a fleet.

Satellite trackers.

CelesTrak [8], N2YO [16] and Heavens-Above [17] publish orbital elements and show where satellites are, using the SGP4 model [3]. They say nothing about a satellite's health.

Table 1: Comparison of existing approaches

Capability	Limit checks	Open MCT	Telemanom [1]	OmniAnomaly [15]	Trackers	Orbital Sentinel
Learned model of normal	No	No	Yes	Yes	No	Yes
Per-alert channel explanation	Channel is the alarm	No	No	Partial	No	Yes (top 3)
Transient-alarm filter	No	No	Pruning only	No	No	Yes (persistence N)
Severity and fleet ranking	No	No	No	No	No	Yes
Live orbital position	No	Plug-in	No	No	Yes	Yes
Operator interface	Yes	Yes	No	No	Yes	Yes
Accuracy reported against labels	No	No	Yes	Yes	n/a	Yes

2.2 Gaps identified and proposed solution

No single tool joins learned detection, per-channel explanation, false-alarm filtering, fleet risk ranking and live orbital awareness in one interface that is honest about how accurate it is. Orbital Sentinel fills that gap with a layered pipeline: replay, then an LSTM detector with the dynamic threshold from [1], then an explanation layer (attribution, persistence filter, severity, ranking) that is this project's own contribution, then a web dashboard and a WebGL globe. Accuracy is measured with the same streaming code that runs live, so the reported numbers describe the system the analyst actually uses.

3. Software Process Model and Project Planning

3.1 SDLC model chosen and justification

Experiment 1 compared six models (Waterfall, V-Model, Incremental, Prototype, Spiral and Agile) and chose an Incremental model with a prototype spike. The reasons still hold after building the system:

- The pipeline splits cleanly into layers that can be built and tested one at a time: ingestion and detection, then explanation, then the dashboard and the globe.
- The requirements were stable once the SRS was written, so heavy change management (as in Spiral or full Agile) was not needed.
- There was exactly one real technical unknown, which detector to use. A throwaway spike (Increment 0) answered it before the design was frozen.
- A working demo was needed early, and each increment produced one.
- It is a one-person project, so the process overhead had to stay small.

Table 2: Increments and when they were delivered

SRS increment	Build phase	Delivered	Finished (git)
Setup	Phase 0 Foundation	Repository, FastAPI backend, database schema, dataset script, CI	5 Oct 2026 (fdd8db9)
Increment 0	Phase 1 Spike	Five candidate detectors compared; ADR 001 and 002	5 Oct 2026 (053dc13)
Increment 1	Phase 2 Ingest + detect	Replay, LSTM, dynamic threshold, lifecycle, log; 81 channels trained	11 Oct (6739e13); report 19 Oct (652d78b)
Increment 2	Phase 3 Explain and prioritise	Attribution, persistence filter, severity, ranking, alert episodes	11 Oct 2026 (1d65efa)
Increment 3a	Phase 4 Dashboard	SSE live updates, health panel, admin, evaluation, roles	11 Oct 2026 (7b13976)
Increment 3b	Phase 5 Globe	CelesTrak client, SGP4, globe, five-satellite fleet	12 Oct 2026 (74835db)
Polish	Performance + UI redesign	Pacing, CPU and globe fixes; redesigned UI	20 Oct 2026 (f2610e5)
Release	Phase 6 Hardening and report	Evaluation runs, full test pass, documentation	In progress (Jira sprints)

Jira was not used during Phases 0 to 5; the work was planned from docs/PHASES.md and recorded in git. Jira was adopted for the final increment, which the course requires. The earlier increments were entered into Jira afterwards from the plan and the git history, with each completed item citing its commit, so the whole project is traceable in one place. The remaining work (evaluation, testing, documentation) was planned and run as two short Jira sprints. Section 7.1 shows the project in Jira.

3.2 Project schedule

The chart below uses real dates. End dates of Experiments 1, 4 and 5 are the dates on those documents, and each phase's end date is its git commit date. Start dates, and the Experiment 3 and 9 dates, are approximate.

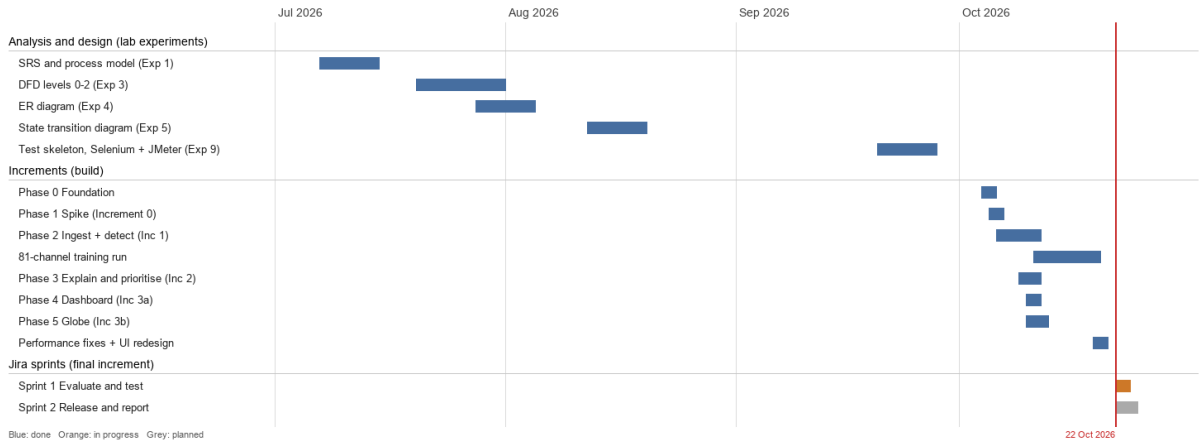


Figure 1: Project schedule (Gantt chart) from the lab documents, git history and Jira sprints

3.3 Risk analysis

Table 3: Risks planned for and what happened

Risk	Likelihood / impact	Mitigation planned	What actually happened
LSTM does not beat simple baselines	Medium / High	Phase 1 spike compared five candidates first	LSTM won clearly: F1 0.79 against 0.51 for the best simple baseline
CPU training too slow	Medium / Medium	Small model (100 steps, 80 units), resumable training	About 5.7 h of CPU because another program was loading the laptop; finished
CelesTrak unavailable	Medium / Medium	2 h cache, back-off, bundled TLE snapshot	Happened (TLS timeouts). The globe kept working after fix B-07
Only two spacecraft in the dataset	Certain / Medium	Proxy satellites, clearly labelled (ADR 004)	Applied to four of the five satellites
Tuning on test data inflates metrics	Medium / High	Separate tuning and reporting halves	Spike figure (0.79) was optimistic; re-measured honestly on 81 channels (0.683)
SQLite write contention	Low / Medium	WAL mode, short transactions	Happened (B-08) and was fixed
Laptop performance during the demo	Medium / High	Paced replay, production build	Happened (B-09) and was fixed with 1 window/s pacing and 2 PyTorch threads
Deadline	High / High	Incremental delivery, demo-able core first	Core finished before the UI polish; final work run as two short sprints

3.4 Effort estimation (basic COCOMO, organic mode)

The delivered source is about 8.9 KLOC (backend about 4.0, frontend about 4.2, spike about 0.7; tests and generated UI components excluded). Basic COCOMO in organic mode gives:

```
Effort = 2.4 x (8.9)^1.05  ~= 23.8 person-months
Time   = 2.5 x (23.8)^0.38 ~=  8.3 months
```

The real effort was far lower: one student over about three and a half months, including the lab documents. COCOMO's 1981 calibration assumes code is written from scratch, while this project leans on large libraries (PyTorch, FastAPI, Next.js, globe.gl). The figure is shown as an illustration of the method, not as a claim about this project.

4. Requirements Analysis

4.1 Functional requirements

Table 4: Functional requirements and their status

ID	Requirement	Status	Implemented in
REQ-1	Load the dataset; replay it as a time-ordered stream per satellite	Done	pipeline/ingest.py (ReplaySource)
REQ-2	Normalise; segment into fixed windows	Done	ingest.py (Normaliser, Windower)
REQ-3	Score each window with a trained baseline (LSTM + dynamic threshold)	Done	pipeline/lstm.py, detect.py
REQ-4	Label a window anomalous when its score exceeds the threshold	Done	detect.py (score > 1)
REQ-5	Compute precision, recall and FPR against ground truth	Done, measured	evaluation/metrics.py, baseline.py
REQ-6	Skip or interpolate malformed values without stopping	Done	ingest.py (repair_series)
REQ-7	Attribute each anomaly to the responsible channel(s)	Done	explain/attribution.py
REQ-8	Persistence filter: alert only after N consecutive windows	Done	pipeline/lifecycle.py
REQ-9	Severity from score and persistence	Done	explain/severity.py
REQ-10	Risk ranking of all satellites	Done	explain/ranking.py, GET /api/ranking
REQ-11	Report FPR with vs without the persistence filter	Done, measured 22 Oct	evaluation/reports.py
REQ-12	Fetch public TLEs; compute positions with SGP4	Done	tracking/celestrak.py, web/lib/sgp4.ts
REQ-13	3D globe with positions updated over time	Done	globe/GlobeCanvas.tsx
REQ-14	Colour each satellite by risk	Done	globe + web/lib/risk.ts
REQ-15	Selecting a satellite opens its health panel	Done	globe detail panel, /satellites/[id]
REQ-16	Risk-ordered alert list with verdict, severity, channel	Done	/dashboard, GET /api/alerts
REQ-17	Drill-down to telemetry plots with the anomaly highlighted	Done	/satellites/[id] (Recharts)
REQ-18	Log every detection with timestamp, score, channel, severity, outcome	Done	DETECTION + ATTRIBUTION tables

4.2 Non-functional requirements

Table 5: Non-functional requirements

Area (SRS)	Target	How it is met	Evidence
Performance 5.1	Score a window and update the panel in about 1 s	CPU inference, 2 PyTorch threads, SSE push	SMAP (18 channels): last window 632 ms, slowest 838 ms (20 Oct)
Performance 5.1	Smooth globe; orbits ready within seconds	SGP4 in the browser, WebGL, TLE cache	First /api/tle 6.7 s with CelesTrak unreachable, 0.04 s after
Safety 5.2	State that it is not an operational tool	Login page, sidebar note, proxy notices, README	Visible in every screen

Security 5.3	Admin functions separated from analyst views	Role checks on every admin route; argon2; signed httpOnly cookie	pytest 403 tests; Selenium TC07, TC10
Reliability 5.4	Malformed input never crashes a session	Gap repair; per-window exception handling	pytest ingest tests (NaN, inf, gaps)
Maintainability 5.4	Stages replaceable, testable with mocks	Python Protocol interfaces, ScriptedDetector	219 tests, mostly mock-based
Portability 5.4	Windows, macOS, Linux	Pure Python + Node, SQLite	GitHub Actions on Ubuntu and Windows (backend job fixed 22 Oct)
Usability 5.4	Readable without training	Colour means risk only, legend, plain labels	Help panel planned (Sprint 2)

4.3 SRS summary

The SRS (Experiment 1, IEEE 830 format [7]) describes three user roles. An Analyst watches the alert list, inspects satellites and acknowledges alerts. An Administrator does everything an Analyst can and also starts and stops monitoring, changes thresholds and imports the dataset. An Evaluator has read-only access to results, logs and metrics. The SRS left three decisions open, all now closed:

Table 6: SRS decisions closed during the build

TBD	Question	Decision	Record
TBD-1	Threshold and persistence N	Dynamic threshold (score > 1); N = 2 chosen by the REQ-11 sweep	docs/adr/003-thresholds.md
TBD-2	Which satellites on the globe	SMAP plus Aqua, Terra, Landsat 9 and Sentinel-2A as labelled proxies	docs/adr/004-fleet-and-proxies.md
TBD-3	Attribution method	Error ranking (15 of 17 correct) over a SHAP-style method (7 of 17)	docs/adr/002-attribution-method.md

4.4 Hardware and software requirements

Table 7: Hardware and software requirements

Item	Used in this project	Minimum to run the demo
Processor	Intel Core i5-13420H, 12 logical CPUs	Any 4-core x86-64 CPU
Memory	16 GB	8 GB (backend about 1 GB while monitoring two satellites)
Disk	About 300 MB for data and models	1 GB free
Operating system	Windows 11	Windows, Linux or macOS
Python	3.12 with FastAPI 0.142, SQLAlchemy 2.1, PyTorch 2.14 (CPU)	Python 3.12
Node.js	22.19 with Next.js 15.5, React 19.1	Node.js 20 or later
Browser	Chrome (WebGL)	Any WebGL-capable browser
Network	Internet for CelesTrak TLEs	Optional: a bundled TLE snapshot is used offline

5. System Design

5.1 System architecture

The system has two data paths that meet only at the dashboard. The health path runs NASA telemetry through ingestion (1.0), detection (2.0) and the explanation layer (3.0) into the database, from where the dashboard (5.0) reads it. The position path takes TLEs from CelesTrak through the tracker (4.0) to the browser, which computes positions itself. The administrator's settings (6.0) are read by the pipeline. The process numbers are the Level-1 DFD's from Experiment 3, so the code and the analysis use the same names.

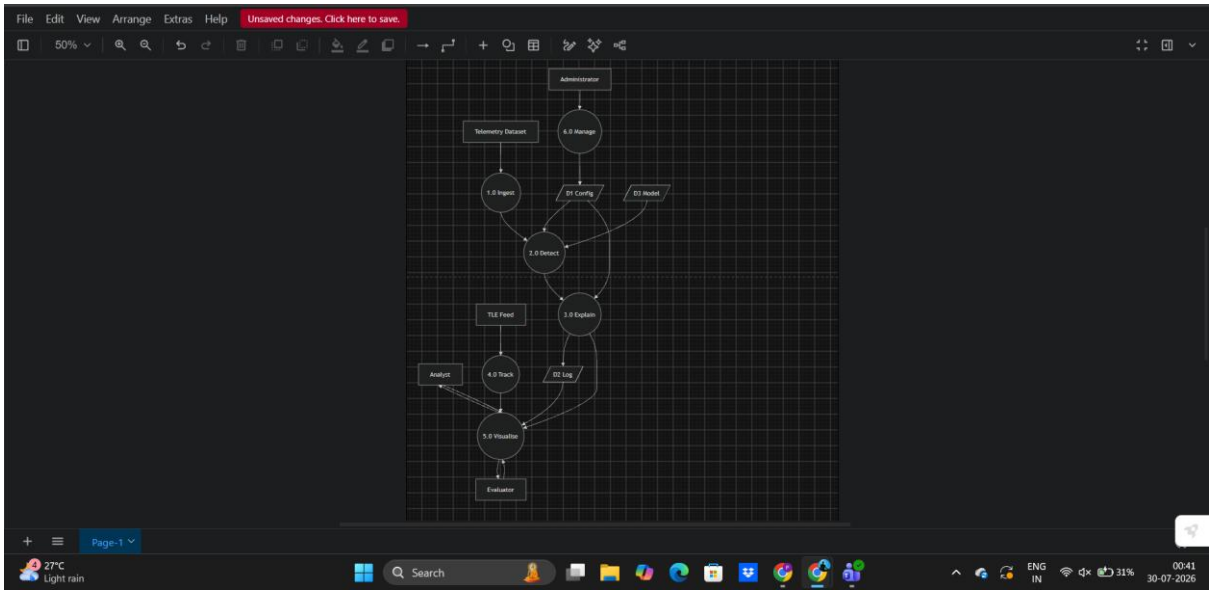


Figure 2: Level-1 data flow diagram (Experiment 3)

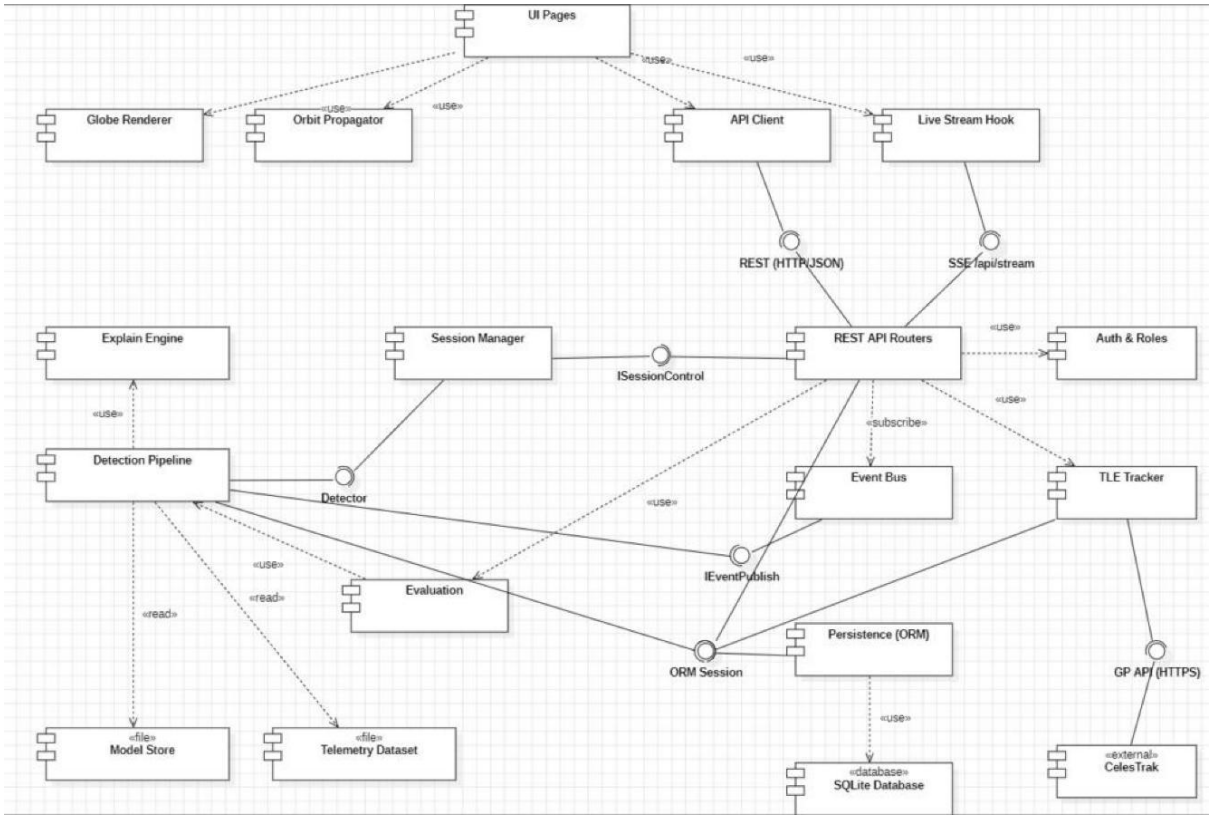


Figure 3: Component diagram: browser, backend, data and external components with their interfaces

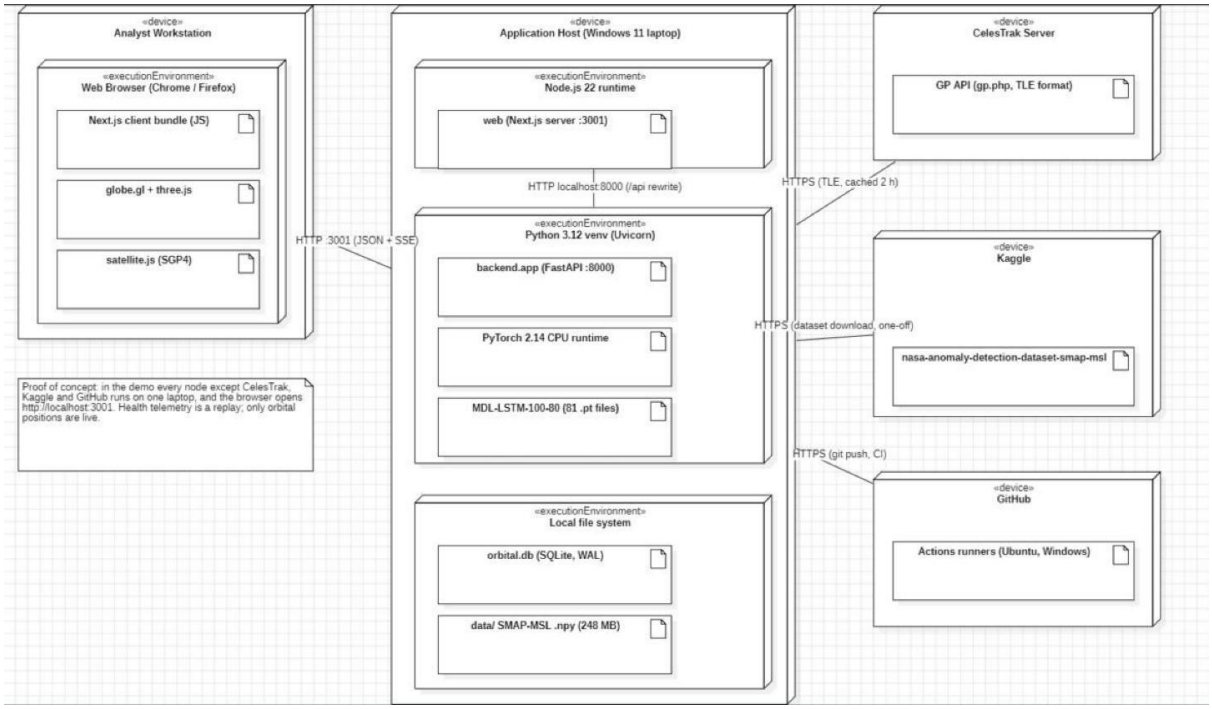


Figure 4: Deployment diagram: nodes, execution environments and communication paths

Table 8: Key architectural decisions

#	Design decision	Reason
AD-1	One FastAPI process holds the ML pipeline and the API	The SRS asks for Python and PyTorch; one process keeps latency simple
AD-2	Next.js and React frontend	Carried over from Experiment 9; the Selenium and JMeter suites target it
AD-3	SQLite through SQLAlchemy and Alembic	The SRS asks for a local database; Postgres works through OS_DATABASE_URL
AD-5	SGP4 runs in the browser	Smooth motion every frame with no server load
AD-6	Server-Sent Events for live updates	One-way push is enough and is simpler than WebSockets
AD-7	Every pipeline stage behind a typed interface	Stages can be replaced and tested with mocks (SRS 5.4)
AD-8	Next.js forwards /api/* to FastAPI	One origin for the browser, so no CORS

5.2 Use case diagram

Four people-facing actors and one external system interact with Orbital Sentinel. User is an abstract actor for anyone signed in. Analyst and Evaluator specialise it, and Administrator specialises both, which matches the server's role checks. Fetching TLEs is included in tracking the fleet; opening the health panel, the telemetry chart and changing the globe speed are optional extensions.

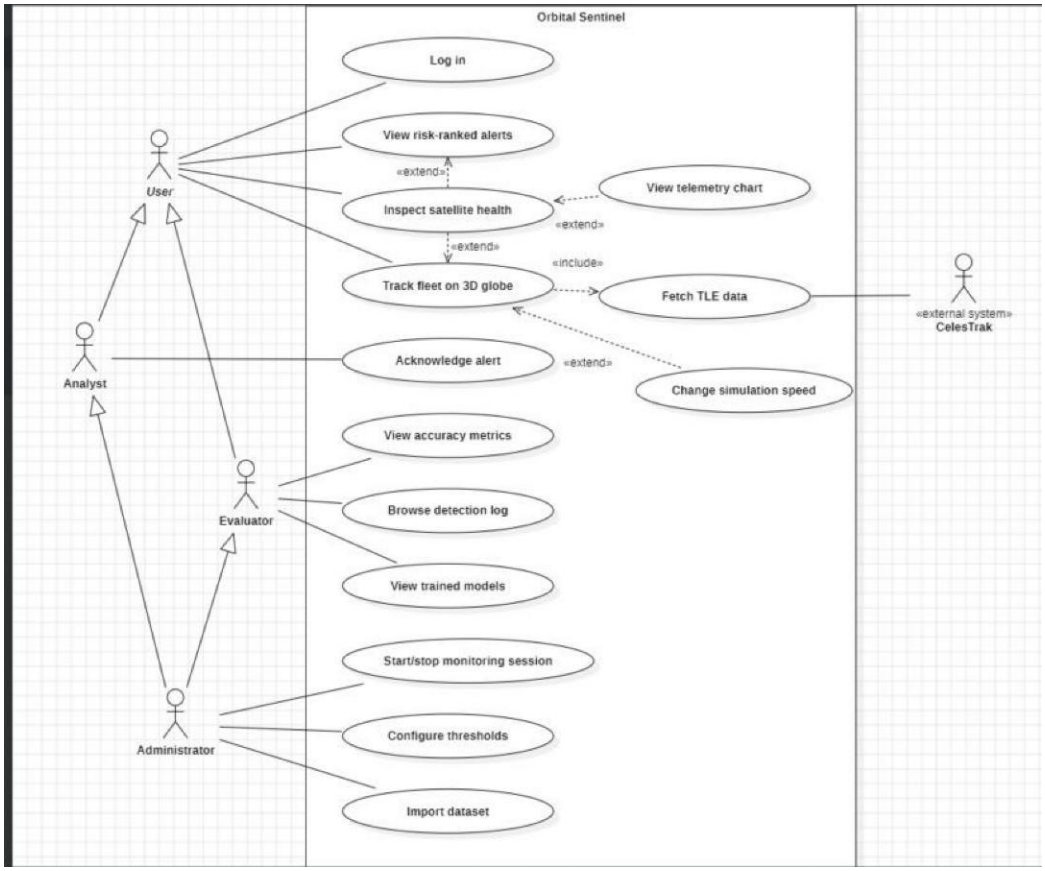


Figure 5: Use case diagram (StarUML, Experiment 6)

5.3 Class diagram

The class model is split into the persistent domain model, whose names follow the ER diagram exactly, and the detection pipeline. MonitoringSession depends on the Detector interface rather than on the LSTM, which is what lets the 219 tests run the whole pipeline with a scripted detector.

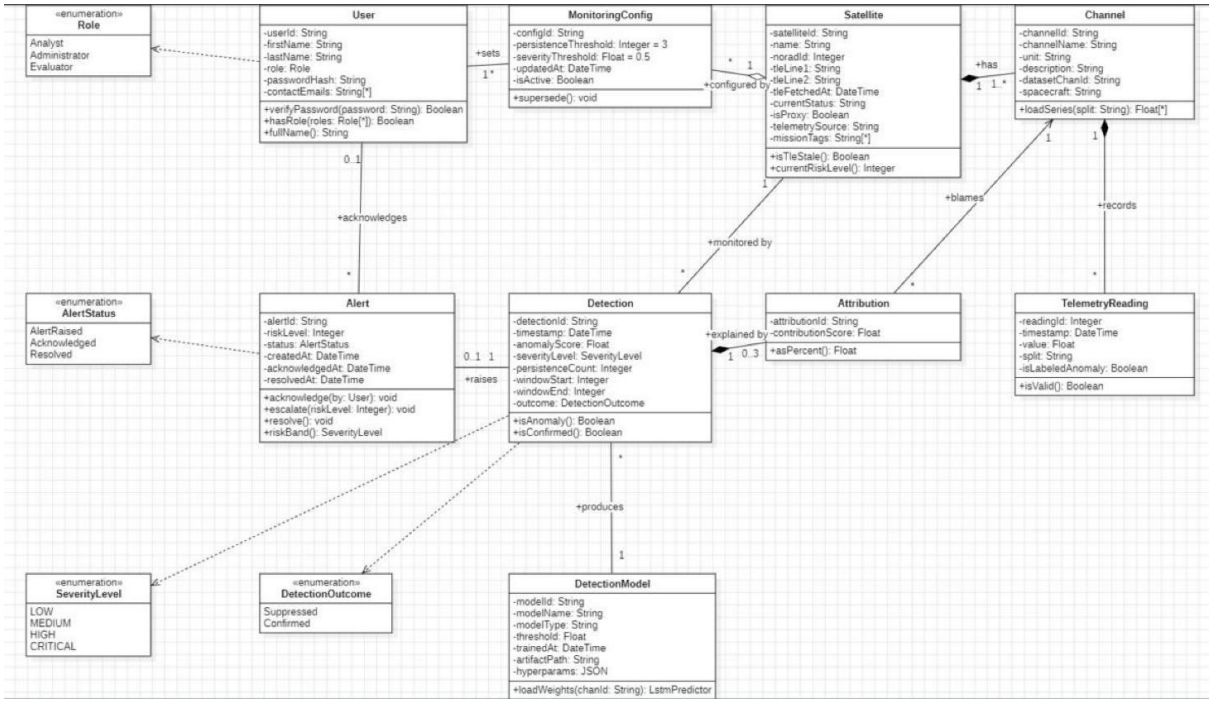


Figure 6: Class diagram: domain model

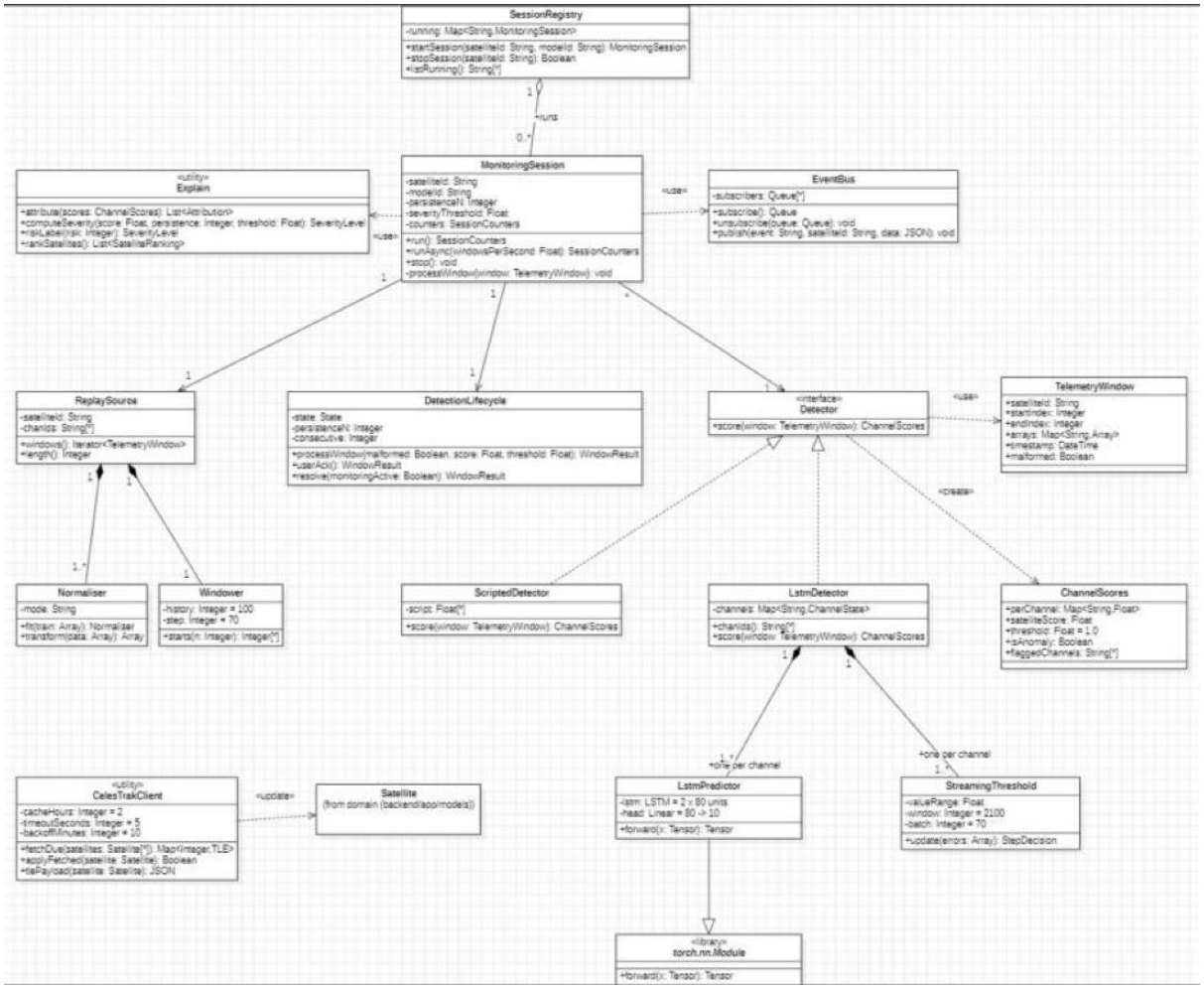


Figure 7: Class diagram: detection pipeline

5.4 Behavioural and interaction diagrams

The detection lifecycle (Experiment 5) has 14 states and 19 transitions, and every one of them is implemented and unit-tested in pipeline/lifecycle.py. Acknowledgement is a human action, so the per-window machine hands over at AlertRaised and the ALERT.status column carries AlertRaised, Acknowledged and Resolved from there. The communication diagram from Experiment 7 shows the messages exchanged while one anomalous window becomes an alert.

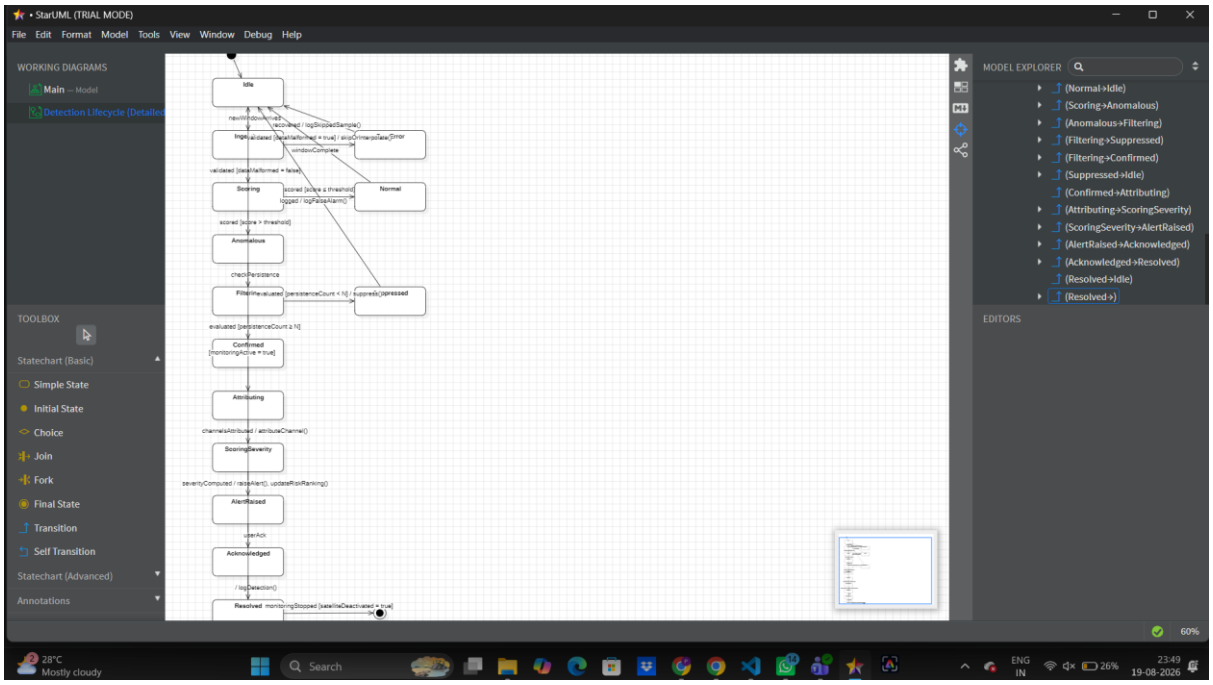


Figure 8: State transition diagram of the detection lifecycle (StarUML, Experiment 5)

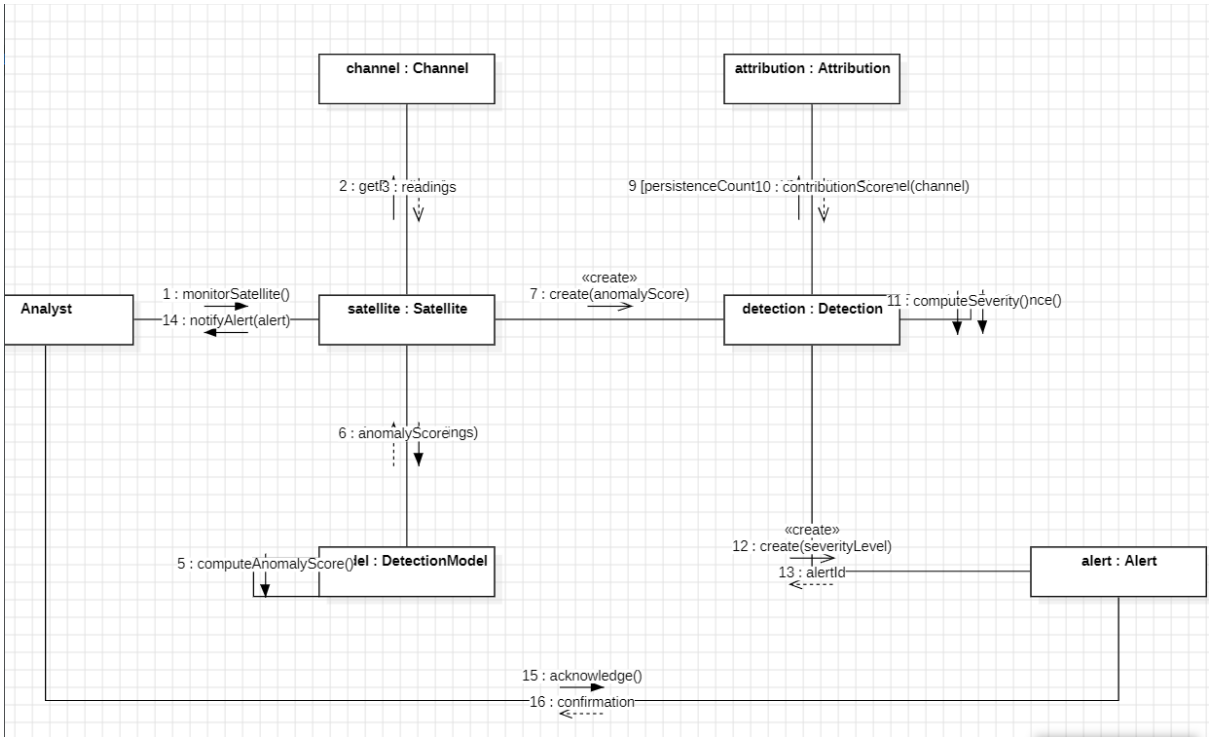


Figure 9: Communication diagram: one window from detection to alert (StarUML, Experiment 7)

5.5 Database design

The database keeps the nine ER entities from Experiment 4 and adds two child tables, satellite_mission_tag and user_contact_email, because those attributes are multivalued and first normal form needs them in their own tables. It runs in WAL mode with a 15 s busy timeout, and the schema is created by an Alembic migration. Derived attributes such as a satellite's current risk level are computed, not stored.

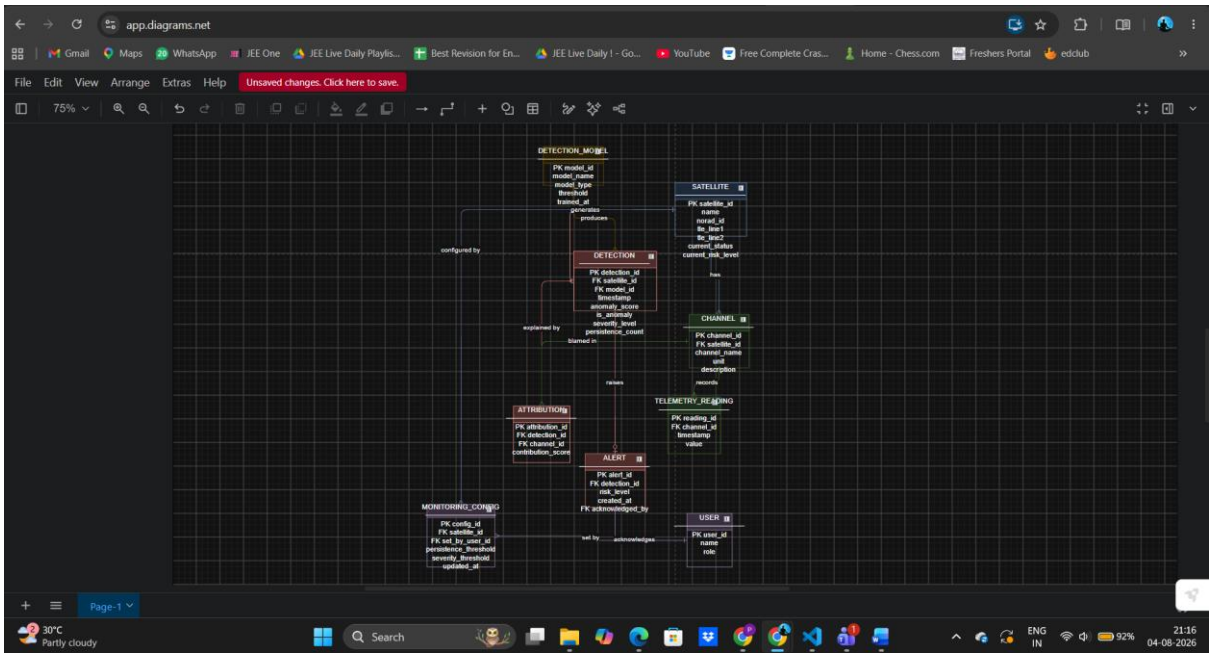


Figure 10: ER diagram (Experiment 4)

Table 9: Physical tables

Table	Key columns (ER names kept)	Added during implementation
satellite	satellite_id, name, norad_id, tle_line1/2, current_status	telemetry_source, is_proxy, tle_fetched_at
satellite_mission_tag	satellite_id, mission_tag	1NF child table
channel	channel_id, satellite_id, channel_name, unit, description	dataset_chan_id, spacecraft
telemetry_reading	reading_id, channel_id, timestamp, value	split, is_labeled_anomaly
detection_model	model_id, model_name, model_type, threshold, trained_at	artifact_path, hyperparams (JSON)
detection	detection_id, satellite_id, model_id, timestamp, anomaly_score, severity_level, persistence_count	window_start, window_end, outcome
attribution	attribution_id, detection_id, channel_id, contribution_score	none
alert	alert_id, detection_id, risk_level, created_at, acknowledged_by	status, acknowledged_at, resolved_at
user	user_id, first_name, last_name, role	password_hash (argon2)
user_contact_email	user_id, contact_email	1NF child table
monitoring_config	config_id, satellite_id, set_by_user_id, persistence_threshold, severity_threshold, updated_at	is_active (history kept)

5.6 UI design

The interface was redesigned on 20 October to look like a working tool rather than a template. Saturated colour is kept for the four risk bands only (critical red, high orange, medium ochre,

low green, grey for nominal), so colour always means risk. The rest is warm off-white with near-black text in the light theme and graphite in the dark theme, with flat 1 px borders, 4 px corners and IBM Plex type, using monospaced figures for IDs and numbers. Loading, empty and error states were designed rather than left blank, and keyboard focus is always visible. Components come from shadcn/ui with its default theme replaced.

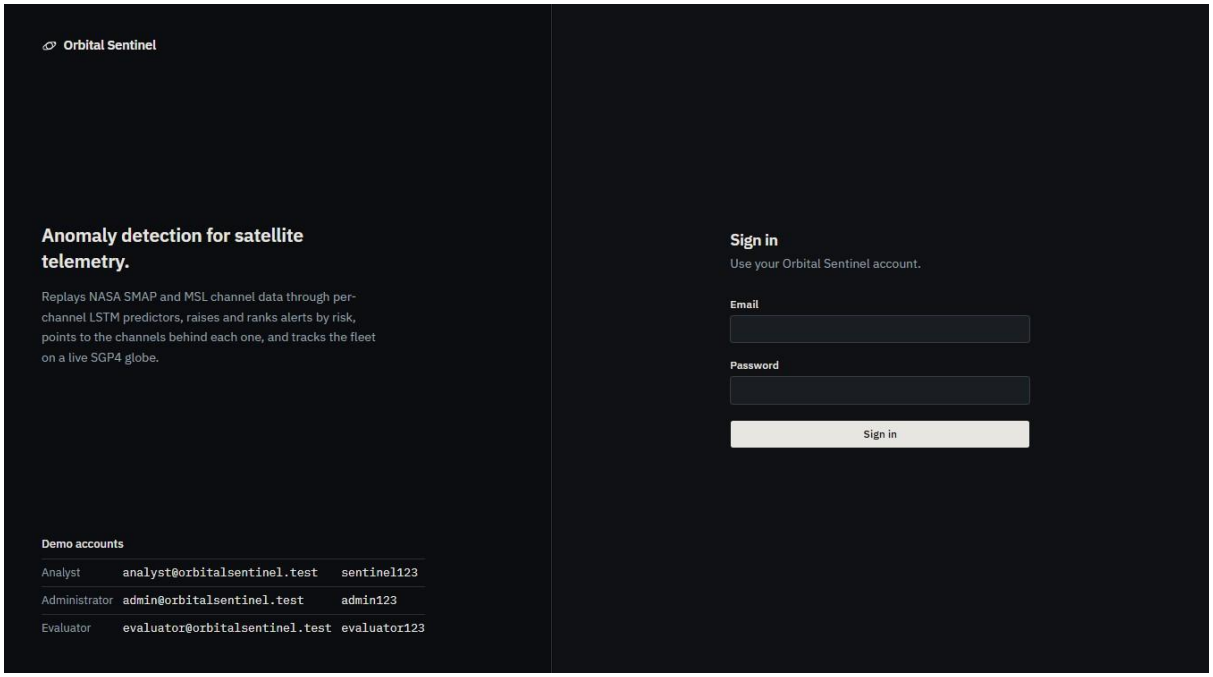


Figure 11: Login page

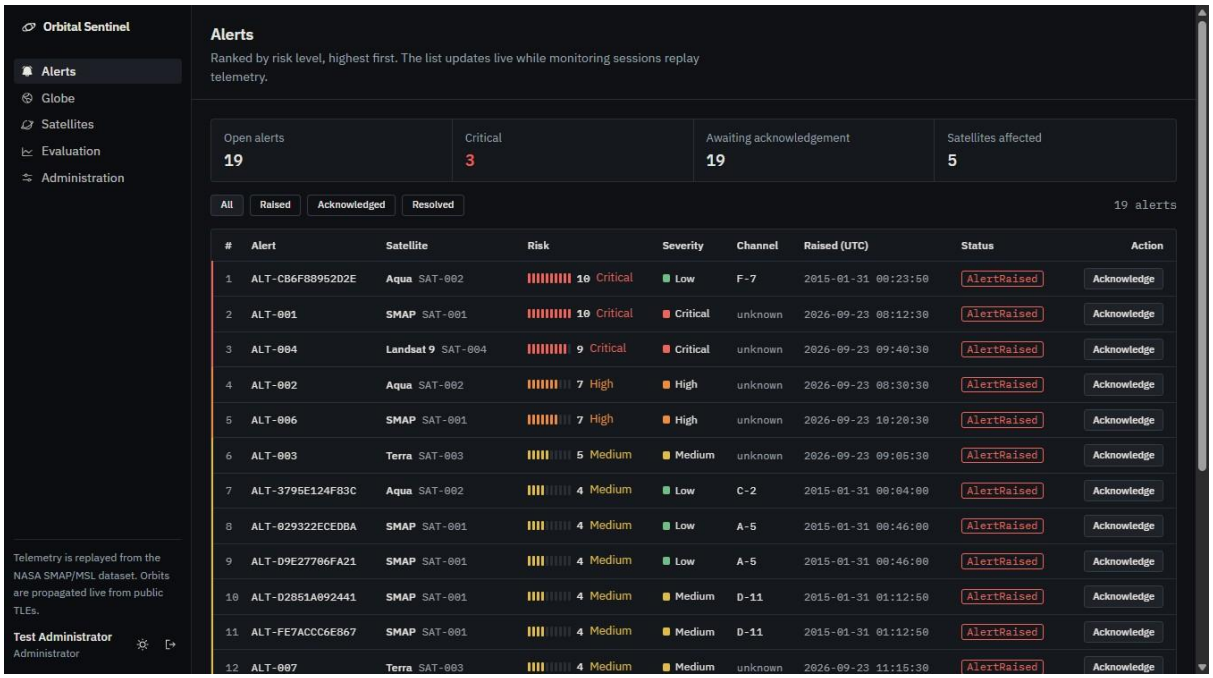


Figure 12: Alerts dashboard: summary strip and risk-ranked alert table

6. Implementation

6.1 Technology stack

Table 10: Technology stack

Layer	Technology (version)	Used for
Backend	Python 3.12, FastAPI 0.142, Uvicorn 0.54	REST API and Server-Sent Events
Data	SQLAlchemy 2.1, Alembic 1.20, SQLite (WAL)	ORM, migrations, storage
Machine learning	PyTorch 2.14 (CPU), NumPy 2.5	Per-channel LSTM models, dynamic threshold
Security	argon2-cffi 25.1, itsdangerous 2.2	Password hashes, signed session cookie
Frontend	Next.js 15.5, React 19.1, TypeScript 5, Tailwind CSS 4	Pages and routing
UI components	shadcn/ui on Radix UI, Recharts 2, Phosphor icons	Tables, controls, telemetry chart
3D and orbits	globe.gl 2.46, three.js 0.186, satellite.js 5	WebGL globe, SGP4 propagation
Testing	pytest, Selenium IDE, Apache JMeter 5.6.3	Unit, functional UI, load tests
Process	Git, GitHub Actions, Jira, StarUML	Version control, CI, planning, UML

6.2 Module description

Ingestion (pipeline/ingest.py). ReplaySource loads each channel's test split, repairs short gaps (up to 5 samples are interpolated, longer gaps are skipped and counted) and yields TelemetryWindow objects of 70 new points with 100 points of history, in time order, as if the data were arriving live.

Detection (pipeline/lstm.py, detect.py). One LSTM per channel (2 layers of 80 units) predicts the next values. The prediction error is smoothed with an exponentially weighted moving average, and StreamingThreshold applies the non-parametric dynamic threshold of [1] to each new batch of 70 errors, looking back over the last 2,100. A channel's score is its peak error divided by the chosen threshold, so a score above 1 means anomalous; the satellite score is the highest channel score.

Lifecycle (pipeline/lifecycle.py). DetectionLifecycle is the Experiment 5 state machine. It counts consecutive anomalous windows and marks a detection Suppressed until the count reaches N, then Confirmed.

Explanation (pipeline/explain/). attribute() ranks the flagged channels by score and stores the top three with their share of the total. compute_severity() combines how strong (score) and how long (persistence) an anomaly is into a 1 to 10 risk level and a band. rank_satellites() orders the fleet by each satellite's highest open alert.

Monitoring service (services/sessions.py, pipeline/session.py). An administrator starts a session per satellite. Each session is an asyncio task that paces the replay at one window per second, runs inference in a worker thread with PyTorch capped at two threads, writes detections, attributions and alerts, and publishes events.

Live updates (services/events.py, api/stream.py). An in-process publish/subscribe bus feeds /api/stream as Server-Sent Events: detection, alert.created, alert.updated and ranking. The browser coalesces refetches to at most one per second so a burst of events cannot flood the API.

Tracking (tracking/celestrak.py). Fetches TLEs from the CelesTrak GP API in parallel with a 5 s timeout, caches them for 2 hours, backs off for 10 minutes after a failure, and falls back to a bundled snapshot so the globe still works offline.

Evaluation (evaluation/). Scores every channel's full test split with the same streaming code used live, and reports event-level precision, recall and F1 plus point false-positive rate. reports.py runs the REQ-11 sweep and comparison.

Web application (web/). Next.js pages for login, alerts, globe, satellites, health panel, evaluation and administration share a sidebar shell whose links depend on the user's role. lib/api.ts is a typed client for every endpoint, and the globe propagates each satellite with SGP4 on every animation frame.

6.3 Key code snippets

The full code is in the repository. These short excerpts show the core of the model, the novelty layer, the role check and the globe loop.

LSTM predictor (backend/app/pipeline/lstm.py)

```
class LstmPredictor(nn.Module):
    """Predicts the next HORIZON telemetry values from HISTORY steps of every feature."""

    def __init__(self, features: int):
        super().__init__()
        self.lstm = nn.LSTM(features, HIDDEN, num_layers=LAYERS, batch_first=True,
dropout=DROPOUT)
        self.head = nn.Linear(HIDDEN, HORIZON)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        out, _ = self.lstm(x)
        return self.head(out[:, -1])
```

Severity and risk level (backend/app/pipeline/explain/severity.py)

```
def compute_severity(score, persistence_count, severity_threshold,
                     score_cap=SCORE_CAP, persistence_saturate=PERSISTENCE_SATURATE):
    norm_score = _clamp((score - 1.0) / max(score_cap - 1.0, 1e-9))
    norm_persistence = _clamp(persistence_count / max(persistence_saturate, 1))
    severity_index = norm_score * norm_persistence
    adjusted = _clamp(severity_index / max(severity_threshold, 1e-9))
    risk_level = max(1, min(10, round(1 + 9 * adjusted)))
    return risk_label(risk_level), risk_level
```

Channel attribution (backend/app/pipeline/explain/attribution.py)

```
def attribute(scores: ChannelScores, top_k: int = TOP_K) -> list[tuple[str, float]]:
    flagged = {c: scores.per_channel[c] for c in scores.flagged_channels if c in
scores.per_channel}
    total = sum(flagged.values())
    if not flagged or total <= 0:
        return []
    ranked = sorted(flagged.items(), key=lambda kv: kv[1], reverse=True)[:top_k]
    return [(chan_id, value / total) for chan_id, value in ranked]
```

Role check used by every protected route (backend/app/core/security.py)

```
def require_roles(*roles: Role):
    """Dependency that allows only the given roles (SRS 5.3, 5.5)."""
    def dependency(user: User = Depends(get_current_user)) -> User:
        if user.role not in roles:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "Your role cannot perform this
action")
        return user
    return dependency
```

Globe frame loop, simplified (web/app/(app)/globe/GlobeCanvas.tsx)

```
const tick = () => {
  raf = requestAnimationFrame(tick);
  const now = simTimeRef.current();
  for (const d of datumsRef.current) {
    const s = stateAt(d.satrec, now);          // SGP4 via satellite.js
    if (s) { d.lat = s.lat; d.lng = s.lng; d.alt = s.alt / EARTH_RADIUS_KM; }
  }
  globe.objectsData(datumsRef.current);
};
```

7. Software Tools Used

Five tools carried the project through planning, design, implementation and testing. Each section says why the tool was chosen and what it was used for, with screenshots taken on the project itself.

7.1 Jira Software (planning and tracking)

Jira is Atlassian's issue tracker for agile teams. It was required by the course and was set up on 22 October for the final increment. The project (key OS) uses a Scrum board. Each SRS increment is an epic with its real start and end dates, so the Timeline view doubles as the project's Gantt chart. Each requirement REQ-1 to REQ-18 and seven non-functional requirements are stories, and the 17 fixed and 2 open defects from the bug log are Bug items, each citing the commit that fixed it. Work done before Jira was adopted is marked Done with its date and commit in the item, rather than back-dated. The remaining work was run as two short sprints with story points, so the burndown chart reflects what really happened.

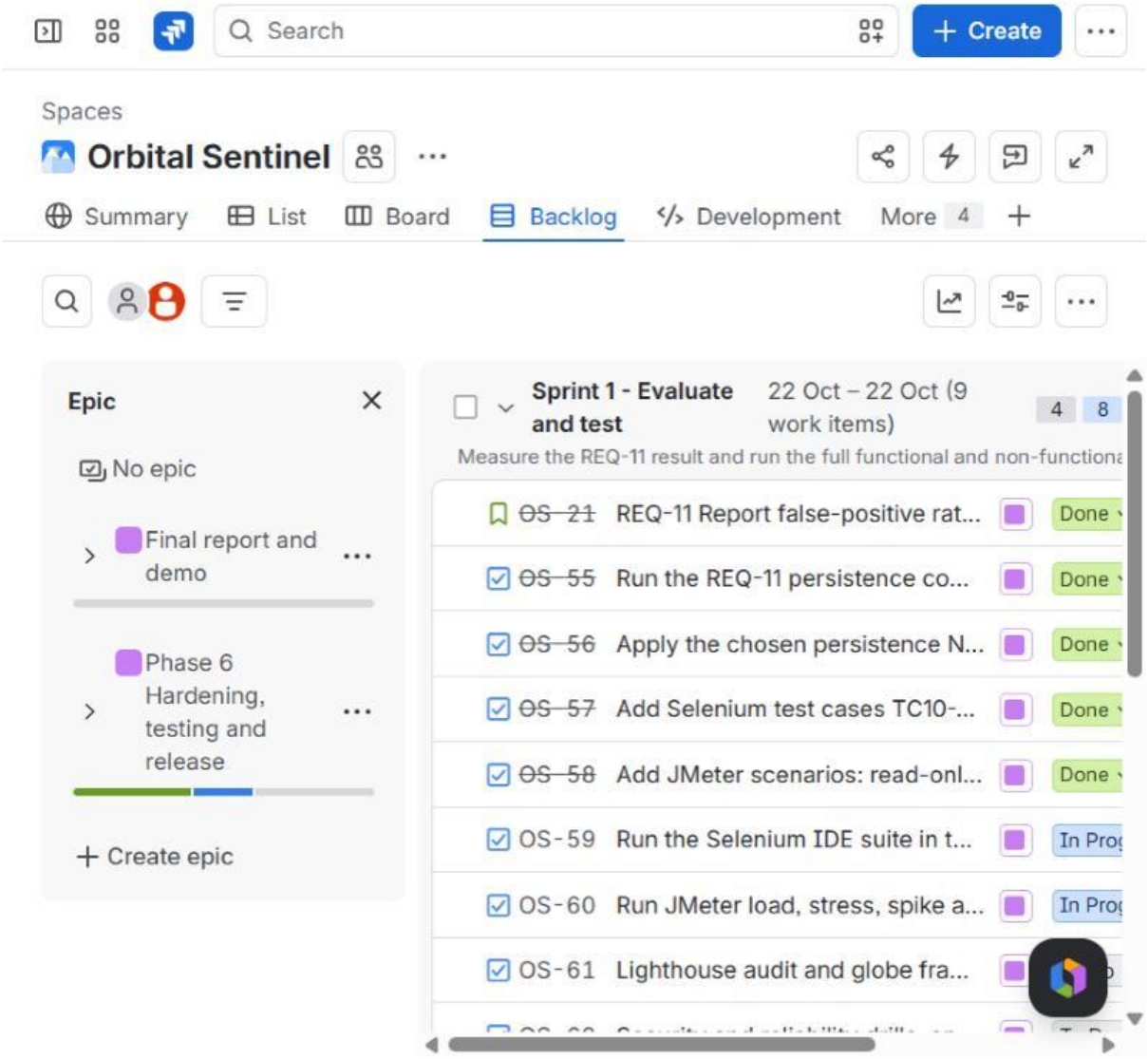


Figure 13: Jira backlog with Sprint 1 and the epics panel

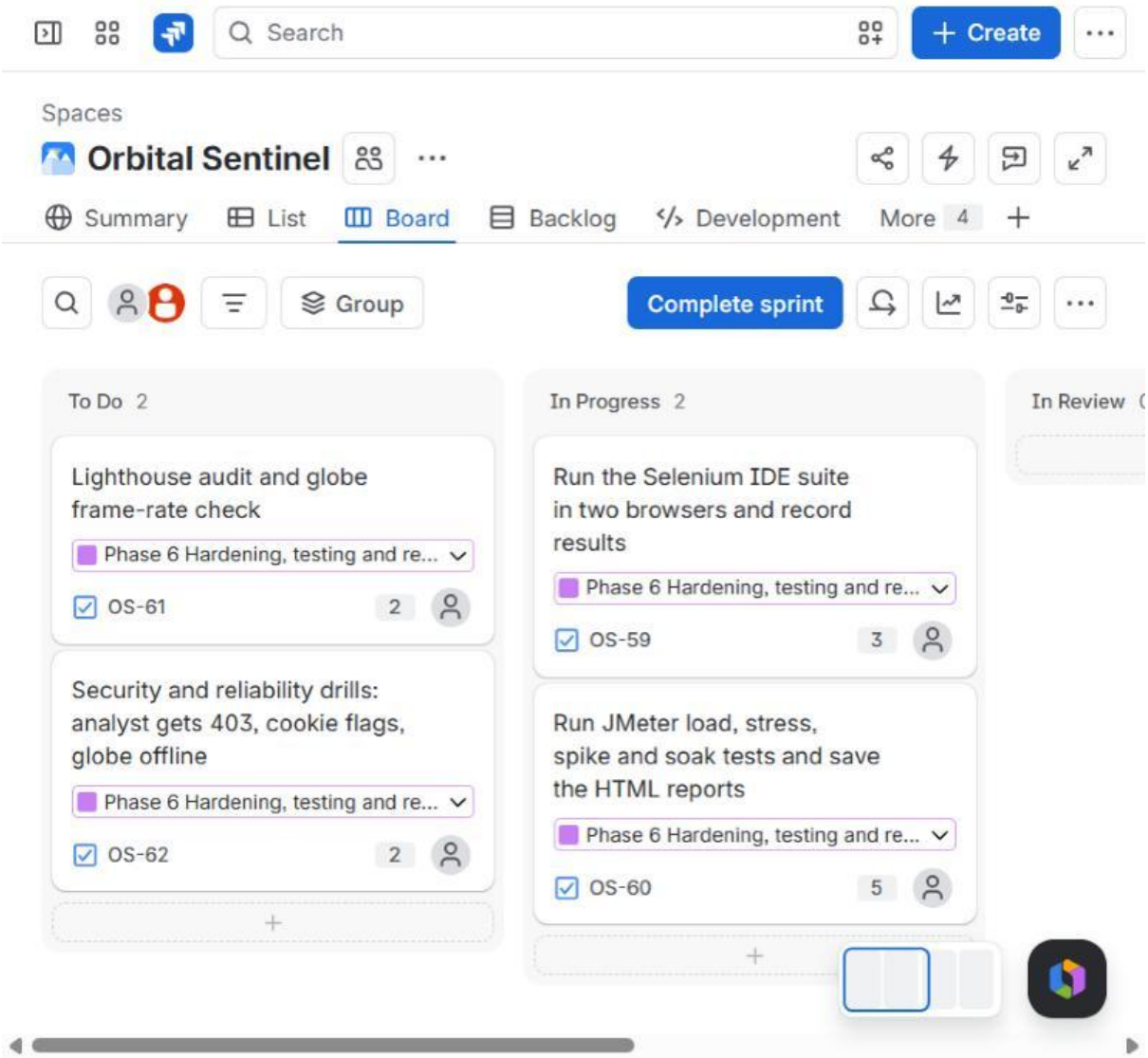


Figure 14: Jira Scrum board for Sprint 1, Evaluate and test

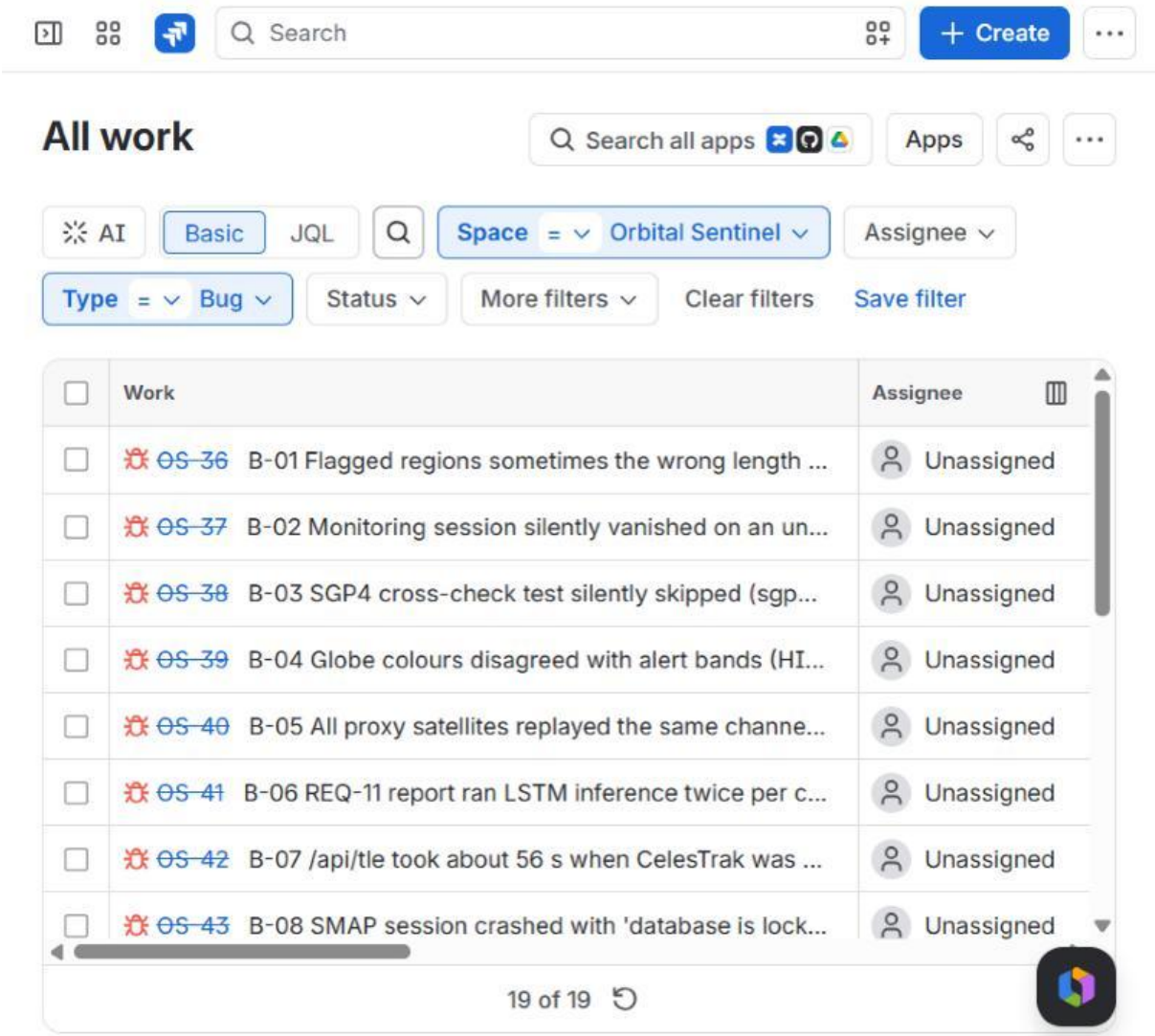


Figure 15: Jira filter type = Bug: the project's defect log

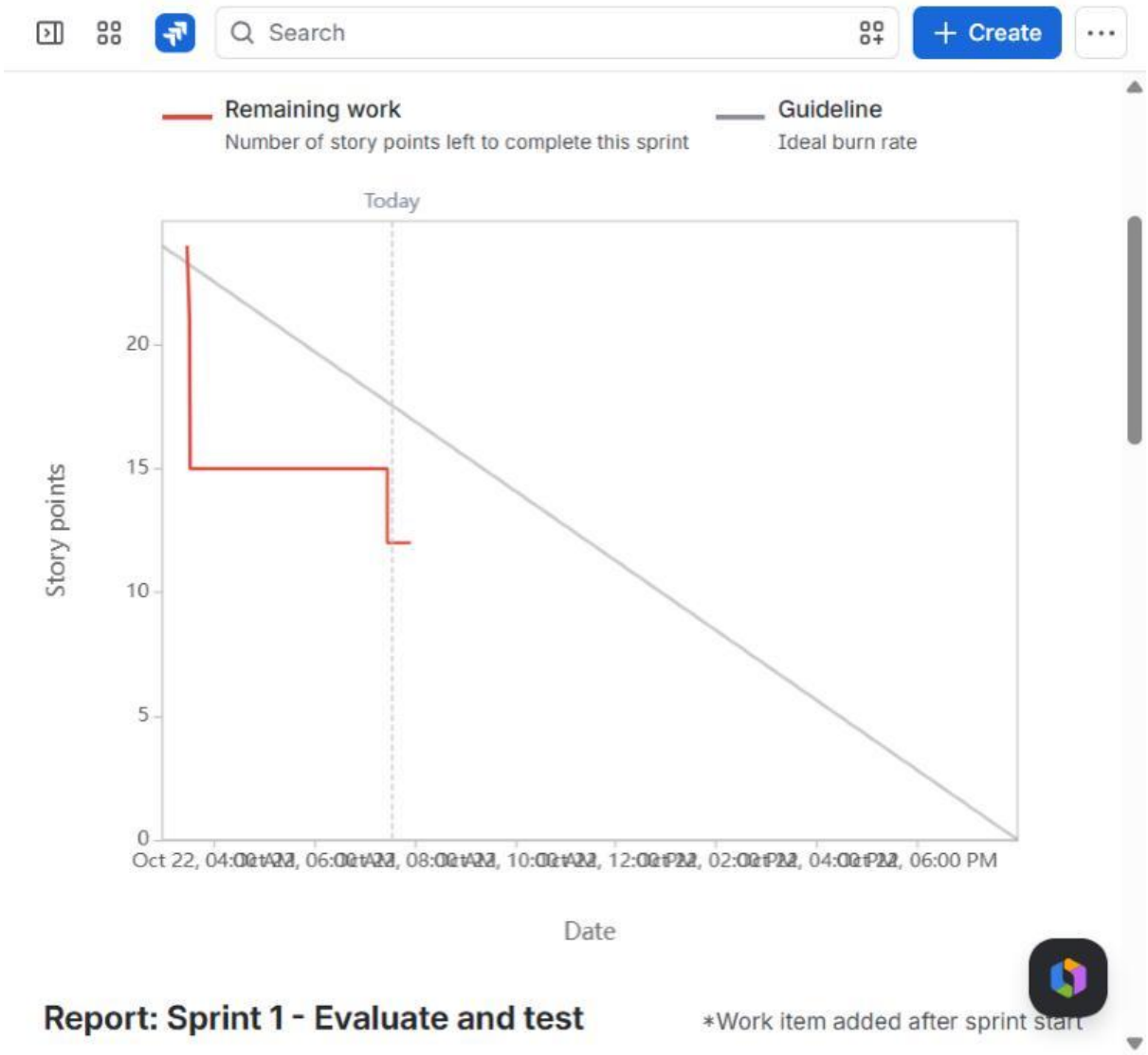


Figure 16: Sprint 1 burndown at 07:35 on 22 October, while the sprint was still running: 12 of 24 story points left

7.2 Git and GitHub with GitHub Actions (version control and CI)

Git tracked every change from the Experiment 9 skeleton to the final increment. The repository is on GitHub at ItsPM25/Orbital_sentinel. Commits are small and describe one change each, for example 'Fix /api/tle stalling and locking SQLite when CelesTrak is unreachable', which made it possible to tie each bug in Jira to the commit that fixed it. GitHub Actions runs .github/workflows/ci.yml on every push to main, on both Ubuntu and Windows: ruff and pytest for the backend, and npm ci, tsc --noEmit and next build for the web app. The large dataset and trained models are kept out of git and documented in the README. Preparing this report showed that the backend CI job had been failing since 7 October without anyone noticing: CI never installed PyTorch, so the test run stopped at the first import, and the fleet tests needed the dataset that CI does not have. Both were fixed on 22 October (bug B-18); in a clean environment without data the suite now gives 208 passed and 11 skipped.

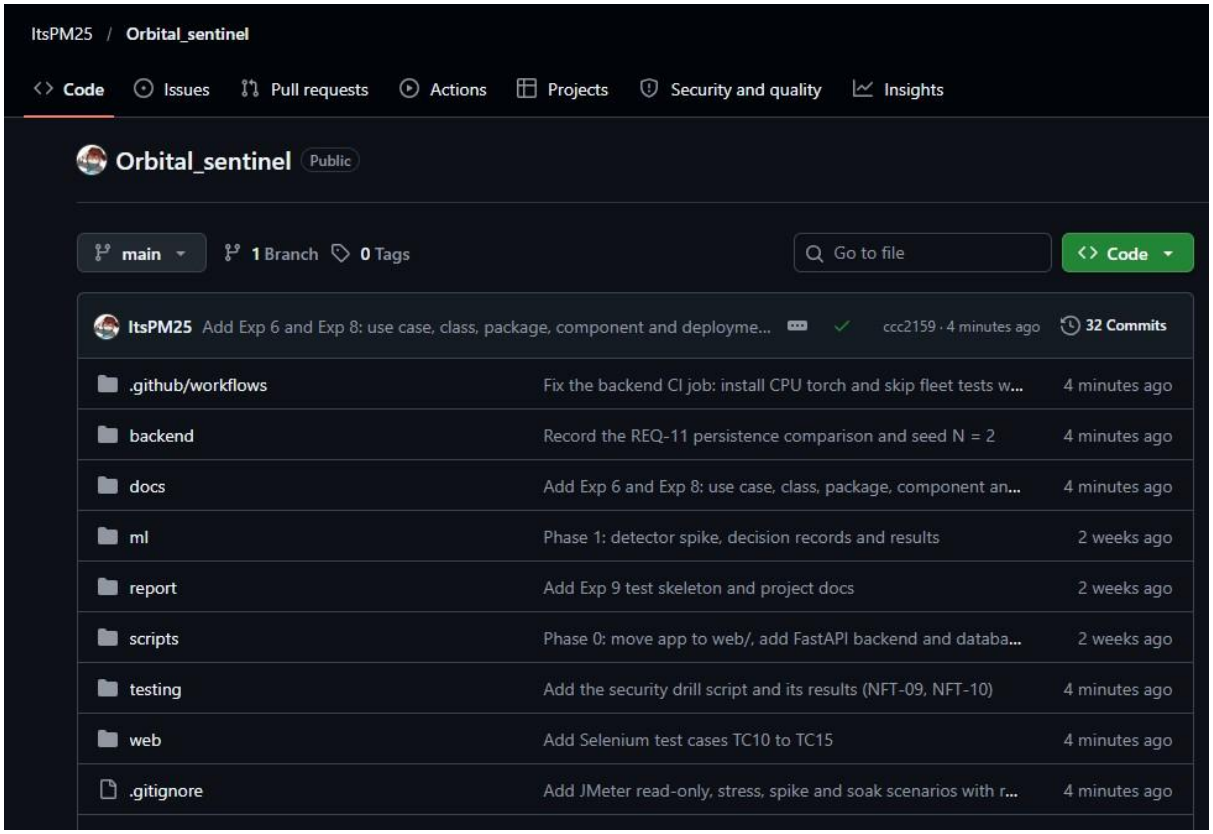


Figure 17: GitHub repository after the 22 October push: the main folders and the latest commit on each

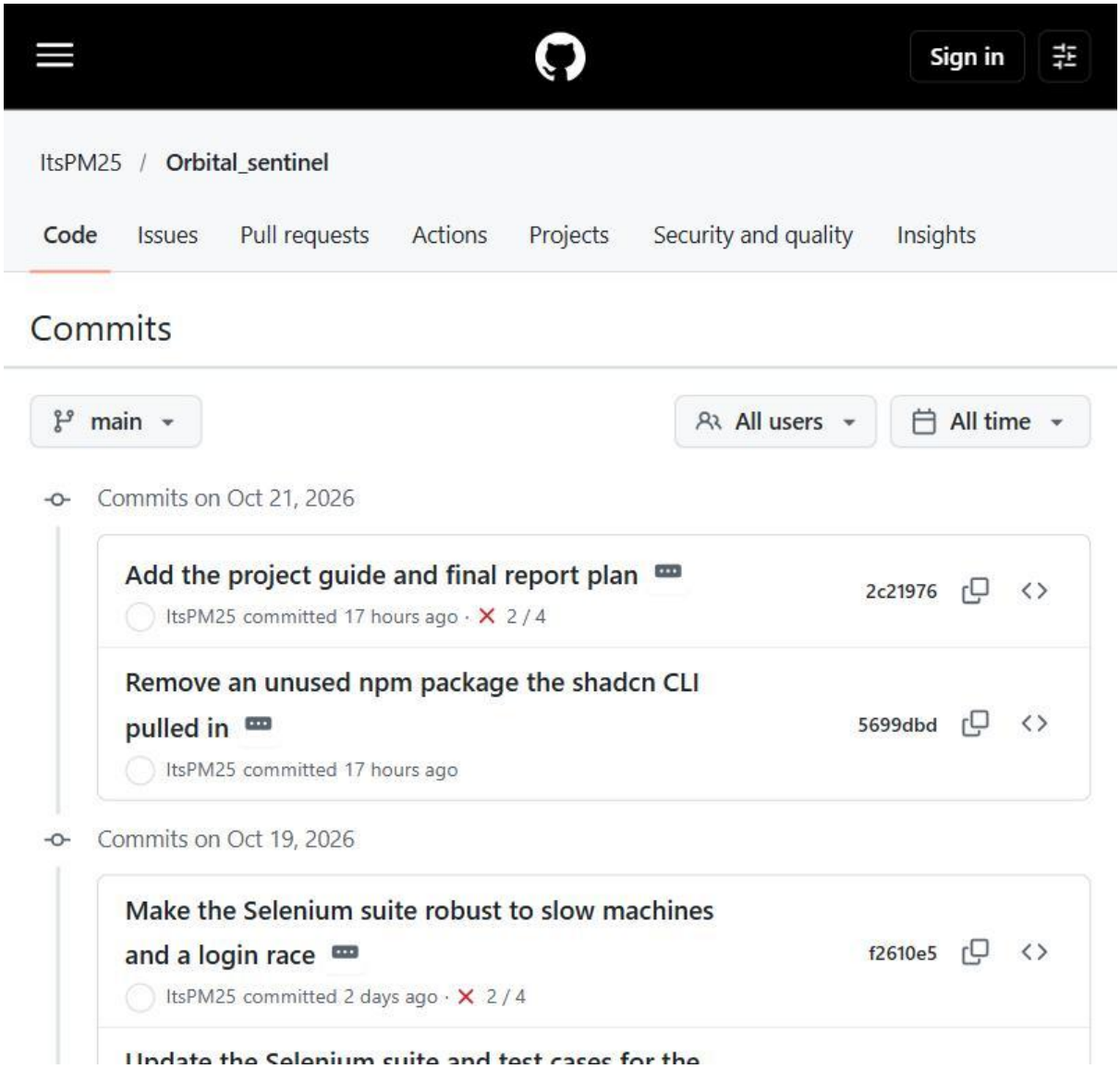


Figure 18: Commit history

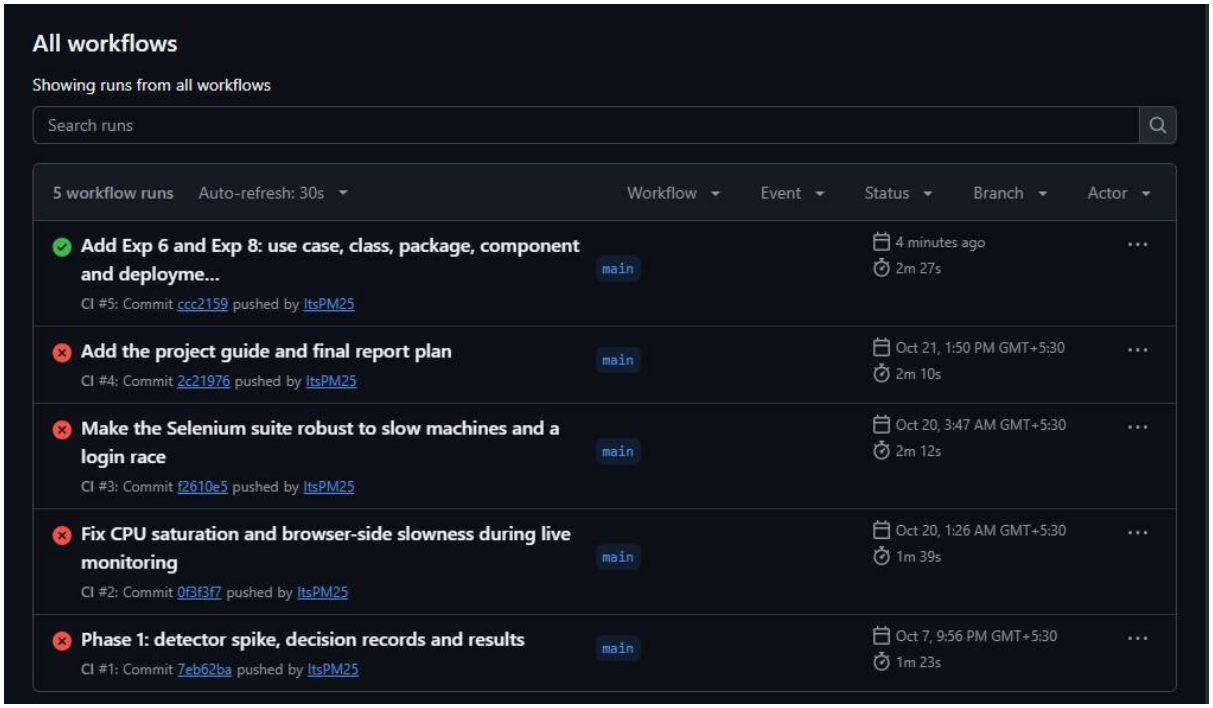


Figure 19: GitHub Actions: the backend job failed on every push from 7 October (B-18); the 22 October push with the fix passed on Ubuntu and Windows

7.3 StarUML (UML modelling)

StarUML was used for every UML model in the project: the state transition diagram (Experiment 5), the communication diagram (Experiment 7), the use case and class diagrams (Experiment 6) and the package, component and deployment diagrams (Experiment 8). Diagram elements for Experiments 6 and 8 were created through StarUML's extension API from a script kept in docs/uml/staruml-extension, so every class and package name matches the code exactly; the layout was then checked in StarUML. The .mdj project files are in docs/uml.

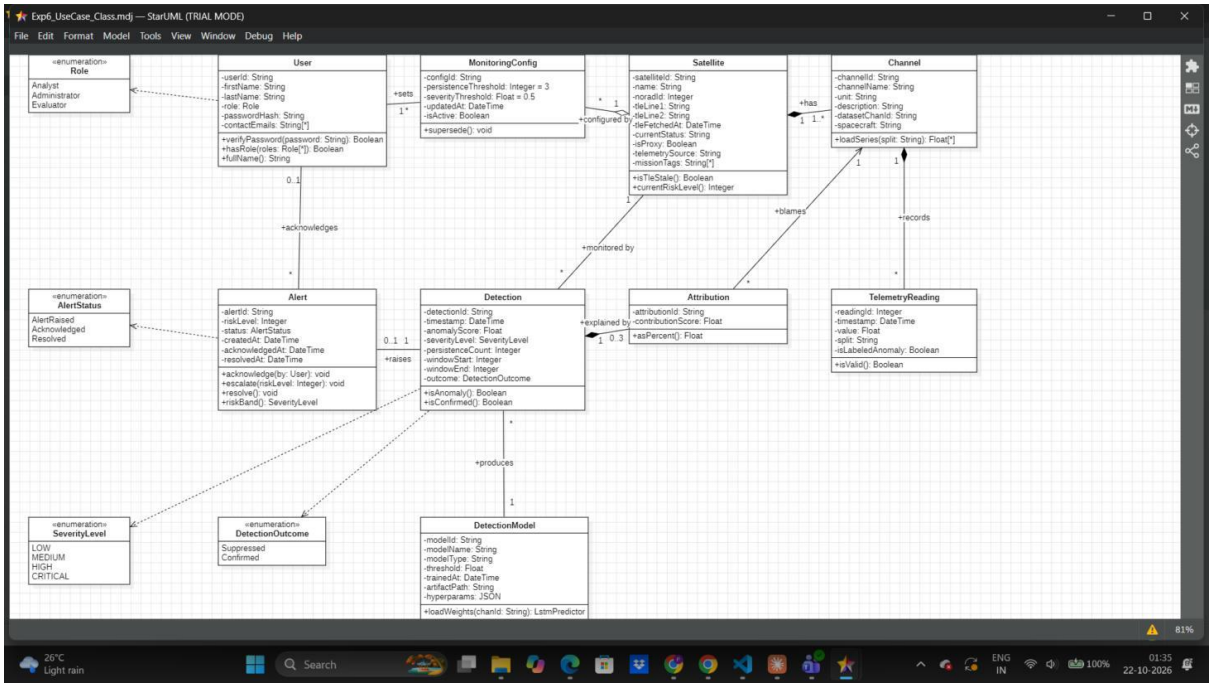


Figure 20: StarUML with the domain class diagram open (system clock visible)

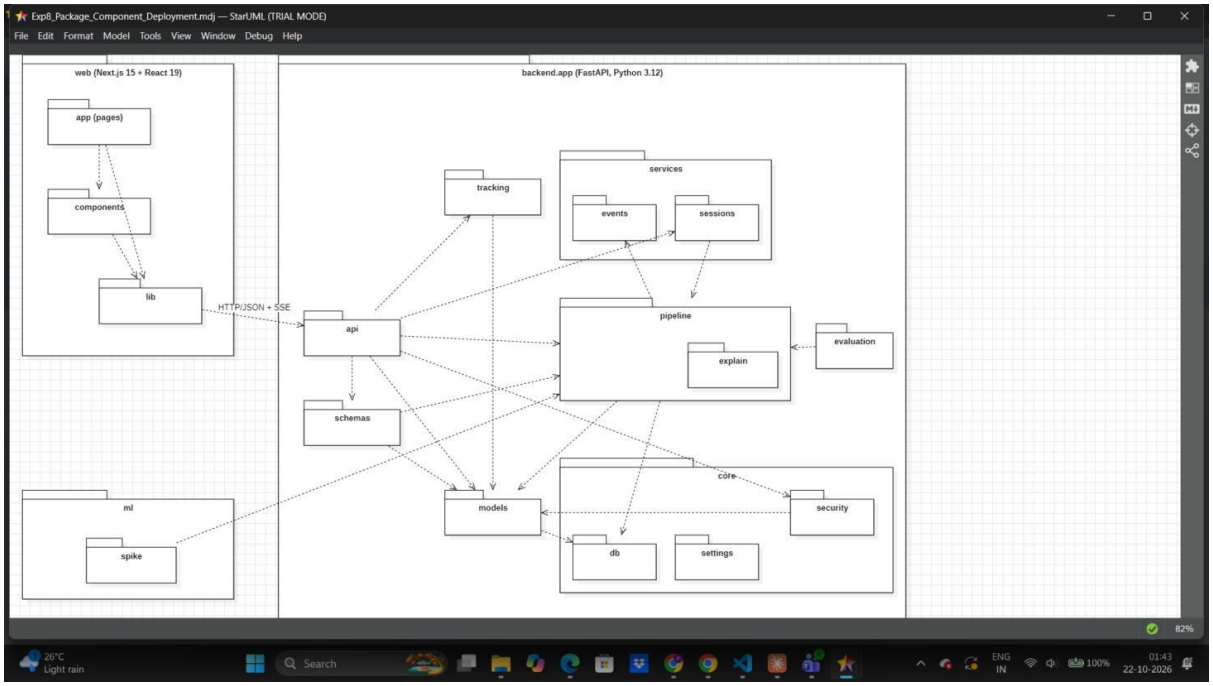


Figure 21: StarUML with the package diagram open (system clock visible)

7.4 Selenium IDE (functional UI testing)

Selenium IDE records and replays browser actions against the running application. The suite testing/OrbitalSentinel.side has 15 test cases that cover login, the alert list and its order, acknowledgement, logout and access control, role separation, drill-down, the globe, administration, the evaluation log and the satellites overview. Each test that needs a known state starts by opening /api/reset, and every locator is an element id kept stable through the UI redesign. The suite was run on 22 October against the production build; the results are in Section 8.2.

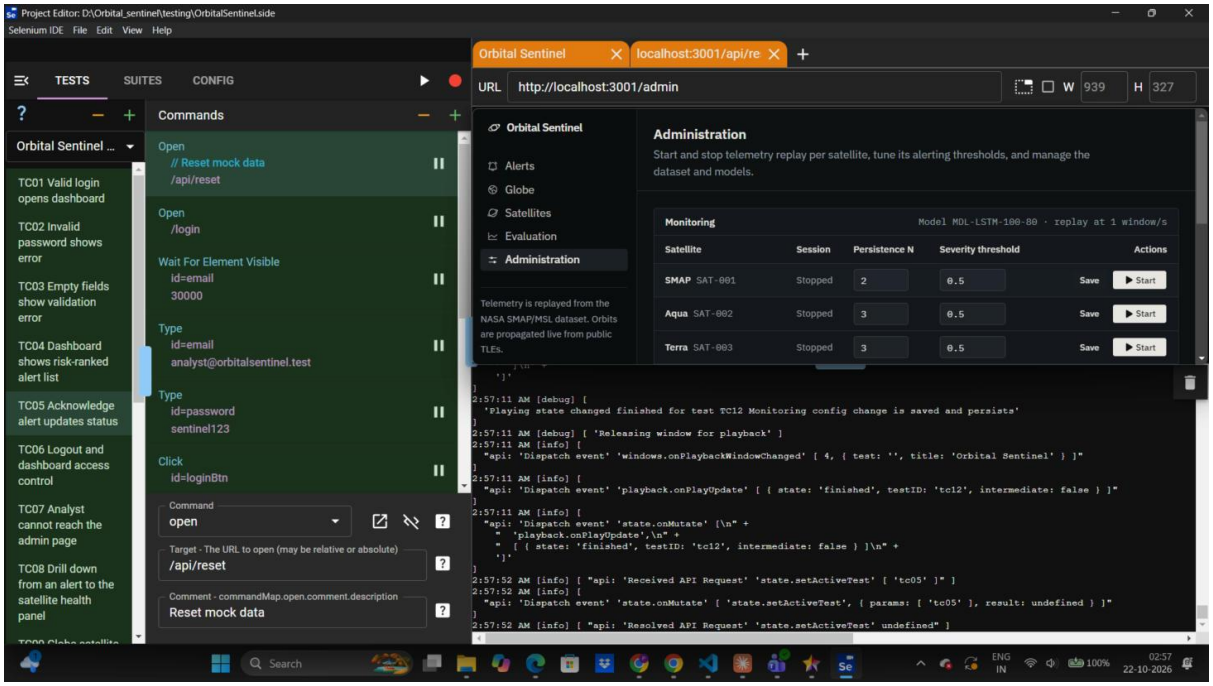


Figure 22: Selenium IDE 4: TC01 to TC08 passed (22 Oct 2026, 02:57)

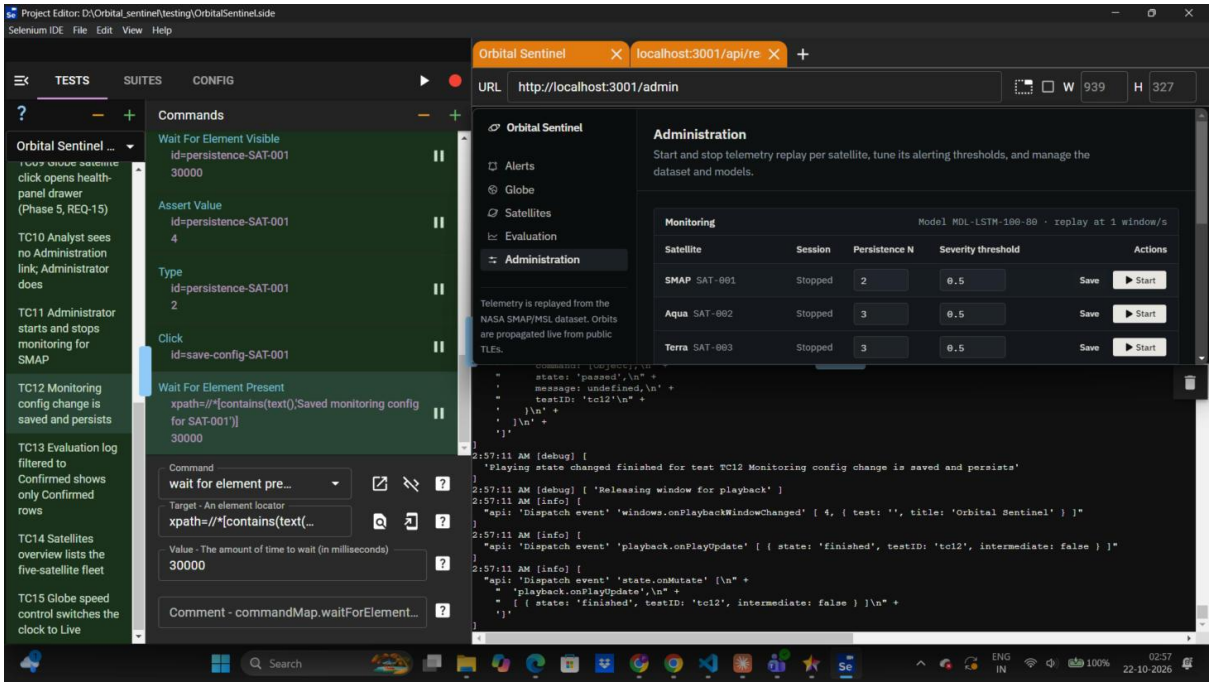


Figure 23: Selenium IDE 4: TC09 to TC15 passed; the Admin page shows SMAP back at N = 2 after TC12

7.5 Apache JMeter (load and performance testing)

JMeter 5.6.3 drives the HTTP API with many virtual users. The plan testing/OrbitalSentinel.jmx has two thread groups. The Analysts group logs in on every loop and then reads alerts, acknowledges an alert, and reads satellites, ranking, telemetry and TLEs. The read-only group logs in once and then repeats the five read requests with 0.5 to 1.5 s of think time, which separates the cost of argon2 password hashing from the cost of reading data. Every request asserts HTTP 200, and the alerts response must contain alertId. All scenarios are run headless by testing/run_jmeter.sh, which produces an HTML dashboard for each one.

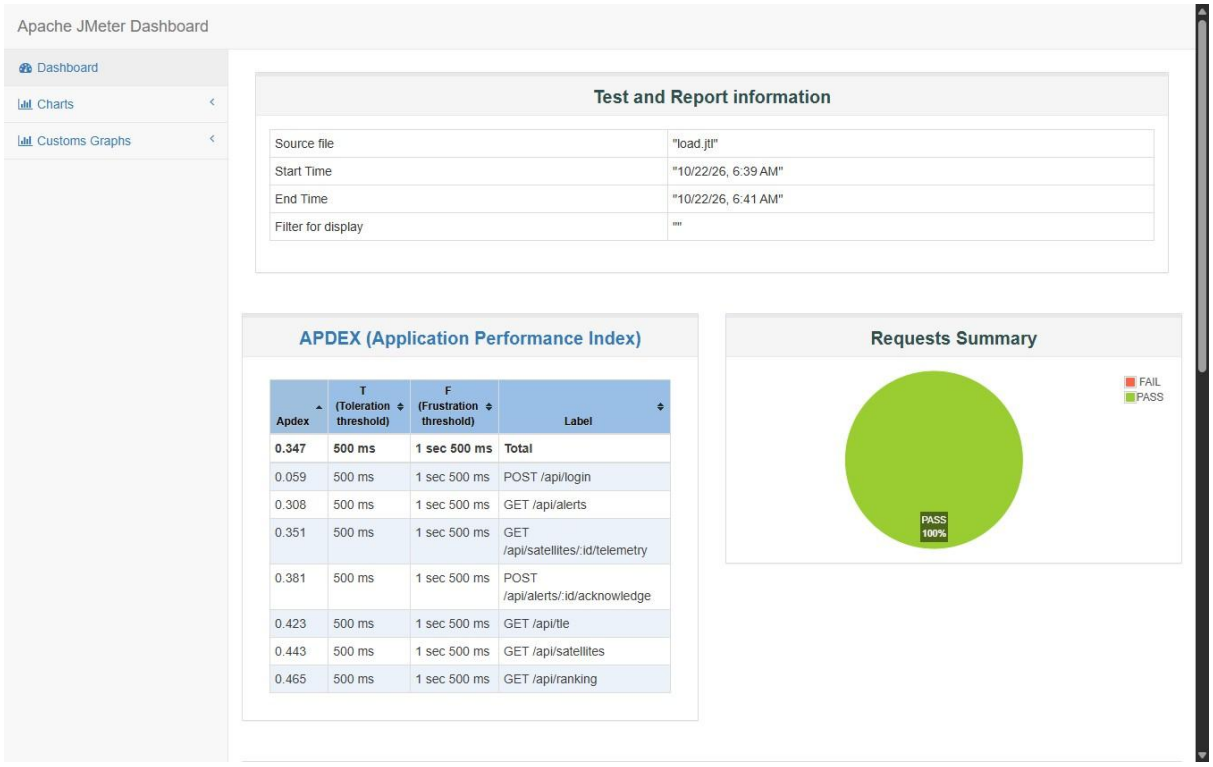


Figure 24: JMeter HTML dashboard for the 50-user load test: no failed requests. The low Apdex comes from login, which runs argon2 on every loop

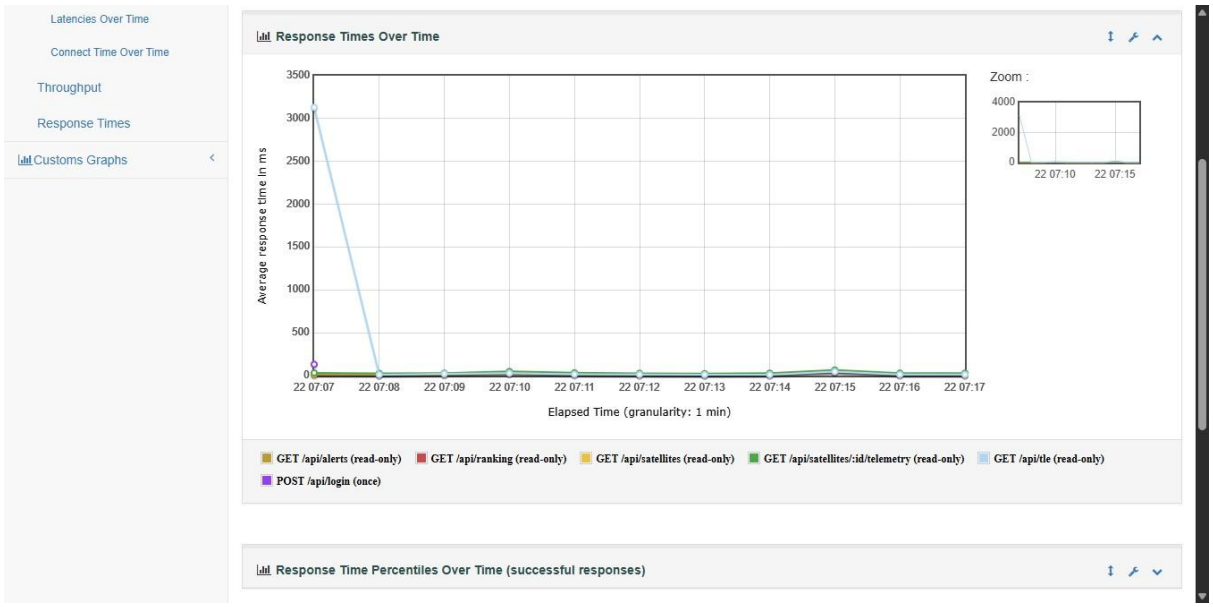


Figure 25: Response times over the 10-minute soak test. The single high point at 07:07 is the first /api/tle call while CelesTrak was unreachable; later calls were served from the cache

7.6 Summary of tools

Table 11: Tools, purpose and phase

Tool	Purpose	SDLC phase	Usage in this project
Jira Software	Project management	Planning, tracking	Epics = increments, stories = REQ-n, bugs = defect log, two sprints, Timeline as Gantt
Git + GitHub + Actions	Version control, CI	Implementation, maintenance	Small described commits; CI with lint, tests and build on Ubuntu and Windows

StarUML	UML modelling	Design	State, communication, use case, class, package, component and deployment diagrams
Selenium IDE	Functional UI testing	Testing	15 test cases against the production build
Apache JMeter	Performance testing	Testing	Load, read-only load, stress, spike and soak scenarios with HTML dashboards
pytest + ruff	Unit and integration tests, linting	Implementation, testing	219 backend tests, including every state-diagram transition
Chrome DevTools	Debugging, performance	Implementation, testing	Found the blank-globe defect (B-11) in the console
VS Code	Editor	Implementation	Editing and running the code

8. Testing

8.1 Test plan and strategy

Testing was planned in layers, from fast automated checks on single functions to slow end-to-end runs through the browser. Every test that needs known data starts from /api/reset, and every test is tied to an SRS requirement where one exists.

Table 12: Test strategy

Level / type	Tool	What is tested	Status (22 Oct)
Unit	pytest	Each pipeline stage with mocks: ingest, threshold, all 19 lifecycle transitions, attribution, severity, ranking, metrics, tracking	219 pass
Integration / API	pytest + FastAPI TestClient	Every endpoint and every role (403s); session pipeline on fixtures	Included in the 219
Functional UI	Selenium IDE	TC01 to TC15 through the browser	15 of 15 pass
Accuracy	Evaluator CLI	Baseline on 81 channels; REQ-11 with and without the filter	Done
Performance	JMeter	Load, read-only load, stress, spike, soak	Run 22 Oct
Security	pytest, curl script	Role enforcement, cookie flags, no secrets in responses	13 of 13 drill checks pass
Reliability	pytest, drills	Malformed data, CelesTrak unreachable	See NFT-11, NFT-12
Portability	GitHub Actions	Ubuntu and Windows	Backend CI fixed 22 Oct (B-18)

8.2 Test cases

Functional test cases (Selenium IDE, run 22 October 2026 on the production build)

Table 13: Functional test cases and results

ID	Input	Expected output	Actual output	Status
TC01	Log in as analyst@orbitalsentinel.test / sentinel123	Dashboard opens; alert table visible; title 'Alerts'	Dashboard opened with the alert table and title	Pass
TC02	Analyst e-mail with wrong password	'Invalid email or password'; stays on /login	Error shown, login button still present	Pass
TC03	Click Login with both fields empty	'Email and password are required'	Message shown	Pass
TC04	Log in; read the alert table after /api/reset	'8 alerts'; row 1 ALT-001, risk 10, Critical; row 8 ALT-005	As expected	Pass
TC05	Click Acknowledge on ALT-001	Status AlertRaised changes to Acknowledged; button disabled	Status Acknowledged; button disabled	Pass
TC06	Log out, then open /dashboard directly	Redirected to the login page; no alert table	Login page shown	Pass
TC07	Log in as Analyst, open /admin	Sent back to the dashboard; admin title never shown	Stayed on dashboard	Pass
TC08	Click the satellite name on the top alert	Health panel for SMAP opens	Title 'SMAP' shown	Pass
TC09	Open /globe, click SAT-001 in the fleet list, then close	Detail drawer for SAT-001 opens and closes	Drawer opened and closed	Pass

TC10	Analyst, then Administrator: read the sidebar	No Administration link for Analyst; link present for Administrator	As expected	Pass
TC11	Administrator: Start SMAP, wait, Stop	Status 'Running' with window counters, then 'Stopped'	Running with counters, then Stopped	Pass
TC12	Set persistence N for SMAP to 4, save, reload	Toast 'Saved monitoring config for SAT-001'; value 4 after reload	Saved and persisted; restored to 2	Pass
TC13	Evaluator: Evaluation page, filter Confirmed	Every row's outcome is Confirmed	Only Confirmed rows	Pass
TC14	Open /satellites	Five rows; SAT-001 present; no sixth row	Five rows	Pass
TC15	Open /globe, click Live	Caption changes from 'Simulated at 60x, UTC' to 'Live, UTC'	Caption changed	Pass

All 15 cases passed when run one by one. Two attempts to run the whole suite at once in Selenium IDE 4 stopped early: the first stalled after TC02, and in the second TC02 timed out because the IDE kept playing back in the browser tab left on /api/reset. The application behaved correctly in both cases; this is recorded as a tool issue (T-02 in Section 8.3). The cross-browser run (Chrome and Firefox, NFT-13) is scheduled for the final pass.

Non-functional test cases

Table 14: Non-functional test cases

ID	Type	Configuration	Pass criterion	Result	Status
NFT-01	Load	50 users, 10 s ramp, 10 loops, login every loop	0% errors; average GET under 1 s	3,500 requests, 0.00% errors, mean 1608 ms (login dominates)	Pass
NFT-01b	Load (read-only)	50 users log in once, 20 loops of 5 GETs	0% errors; mean under 1 s	5,050 requests, 0.00% errors, mean 85 ms, 90th pct 211 ms	Pass
NFT-02	Stress	100 users / 30 s ramp / 5 loops; a 200-user run was planned next	Find where errors or latency climb	100 users: 83.40% errors once about 55 users were active (O-03). 200-user run not needed once the limit was found	Limit found: 50 to 60 users
NFT-03	Spike	200 users started within 1 s, 1 loop	System recovers; errors reported honestly	98.36% errors during the burst (O-03); service recovered on its own after it	Fail (O-03); recovers
NFT-04	Soak	20 read-only users for 10 minutes	No rising latency; memory steady	11,574 requests, 0.00% errors, mean 31 ms	See 8.3
NFT-05	Response time per endpoint	From the load and read-only runs	Typical under 1 s	See Tables below	See below
NFT-06	Detection latency	SMAP session, Admin counters	Last and slowest window about 1 s or less	632 ms last, 838 ms slowest (20 Oct, loaded laptop)	Pass
NFT-07	Globe frame rate	DevTools FPS meter, /globe at 60x	Steady near the display rate	Pending (final pass)	Pending
NFT-08	Front-end quality	Lighthouse on /login and /dashboard	Report the scores	Pending (final pass)	Pending
NFT-09	Security: roles	testing/security_drills.sh: analyst and evaluator cookies on POST /api/sessions; no cookie; forged cookie	403 for wrong role, 401 without a valid session	403, 403, 401, 401 as expected; 9 of 9 checks pass	Pass
NFT-10	Security: session cookie	Same script: Set-Cookie flags; login, me, alerts, satellites, ranking and sessions bodies	HttpOnly; no password or hash in any response	HttpOnly, SameSite=Lax, 8 h expiry; no argon2 hash or password field found; 4 of 4 pass. No Secure flag, since the demo runs on plain HTTP	Pass

NFT-11	Reliability: offline	CelesTrak unreachable	Globe still renders from cache	Measured 19 Oct: first /api/tle 6.7 s, then 0.04 s; globe kept working (B-07)	Pass
NFT-12	Reliability: malformed data	pytest test_ingest (NaN, inf, gaps)	Session never stops	15 ingest tests pass	Pass
NFT-13	Compatibility	Selenium suite in two browsers	Same results in both	Chromium (Selenium IDE) done; second browser pending	Pending
NFT-14	Portability	GitHub Actions matrix	Green on Ubuntu and Windows	All four jobs (backend and web, Ubuntu and Windows) green on 22 Oct, run #5; backend had failed since 7 Oct (B-18)	Pass

Table 15: NFT-01 load test (50 users, login every loop): per-request statistics

Request	Samples	Mean ms	Median ms	90th pct ms	Max ms	Errors	Req/s
GET /api/satellites	500	1,225	1,003	1,979	8,381	0.00%	4.2
POST /api/alerts/:id/acknowledge	500	1,376	1,181	2,269	7,234	0.00%	4.2
GET /api/satellites/:id/telemetry	500	1,451	1,317	2,052	10,960	0.00%	4.2
GET /api/tle	500	1,425	1,112	2,082	17,321	0.00%	4.3
POST /api/login	500	2,996	2,629	5,387	9,996	0.00%	4.2
GET /api/ranking	500	1,141	992	1,867	7,184	0.00%	4.2
GET /api/alerts	500	1,642	1,344	2,582	8,645	0.00%	4.2
TOTAL	3,500	1,608	1,249	2,752	17,321	0.00%	29.5

Table 16: NFT-01b read-only load (50 users, logged in once): per-request statistics

Request	Samples	Mean ms	Median ms	90th pct ms	Max ms	Errors	Req/s
GET /api/satellites (read-only)	1,000	63	32	147	780	0.00%	8.6
GET /api/tle (read-only)	1,000	51	25	98	926	0.00%	8.6
GET /api/ranking (read-only)	1,000	57	29	125	786	0.00%	8.6
GET /api/satellites/:id/telemetry (read-only)	1,000	182	93	492	1,120	0.00%	8.6
POST /api/login (once)	50	451	385	835	1,009	0.00%	4.9
GET /api/alerts (read-only)	1,000	54	29	113	899	0.00%	8.6
TOTAL	5,050	85	36	211	1,120	0.00%	41.5

Table 17: NFT-02 stress test (100 users, login every loop): per-request statistics

Request	Samples	Mean ms	Median ms	90th pct ms	Max ms	Errors	Req/s
GET /api/satellites	500	25,465	30,021	30,095	30,513	84.40%	0.5
POST /api/alerts/:id/acknowledge	500	25,073	30,019	30,095	30,314	83.00%	0.5
GET /api/satellites/:id/telemetry	500	26,038	30,023	30,113	30,603	86.20%	0.5
GET /api/tle	500	26,363	30,023	30,113	30,596	87.40%	0.5
POST /api/login	500	23,691	30,017	30,094	30,613	77.20%	0.6
GET /api/ranking	500	25,721	30,021	30,118	30,611	85.20%	0.5
GET /api/alerts	500	24,331	30,019	30,094	30,580	80.40%	0.5
TOTAL	3,500	25,240	30,021	30,104	30,613	83.40%	3.2

8.3 Test results and bug log

Testing found real defects throughout the build. Each one below is a Bug item in Jira with the same number, and each fixed one names the commit that fixed it.

Table 18: Bug log

ID	Found	Symptom	Cause	Fix / commit	How found
B-01	11 Oct	Flagged regions sometimes the wrong length	np.convolve(mode='same') returns the kernel's length when the kernel is longer than the input	6739e13	Unit test
B-02	11 Oct	A monitoring session silently vanished	Unregistered model hit a foreign-key error inside the background task	7b13976	Smoke test
B-03	12 Oct	SGP4 cross-check test was skipped	Reference sgp4 package not installed	db171cb	Review
B-04	12 Oct	Globe colours disagreed with alert bands	Frontend used >= 7 for HIGH, backend >= 6	db171cb	Review
B-05	12 Oct	All proxies replayed the same channels	ADR 004's five-way channel split never implemented	74835db	Review
B-06	19 Oct	REQ-11 report ran twice as long	LSTM inference ran twice per channel	652d78b	Review
B-07	19 Oct	/api/tle took about 56 s	Sequential CelesTrak timeouts, failures not remembered	149493a	Live run
B-08	19 Oct	SMAP session crashed: database is locked	Write transaction held across slow network calls	149493a	Live run
B-09	20 Oct	Laptop slowed during monitoring	Unpaced replay; 12 PyTorch threads per session	0f3f3f7	Profiling
B-10	20 Oct	Globe page leaked GPU work	Async init raced cleanup; about 295 orbit meshes rebuilt per render	0f3f3f7	Code review
B-11	20 Oct	Globe blank	Two three.js versions (0.170 and 0.186) in one scene	77e9888	Browser console
B-12	20 Oct	Request storm during replays	Every SSE event triggered refetches	0f3f3f7	Profiling
B-13	20 Oct	Satellite red but labelled LOW	Ranking used first severity, not escalated risk	4fa6bd4	Visual check
B-14	20 Oct	'8 alerts' test failed after monitoring	/api/reset left replay-generated alerts	5ac68bb	Selenium
B-15	20 Oct	Channel A-1 chart flat	Downsampling dropped short spikes	c3340c1	Visual check
B-16	20 Oct	Selenium TC07 flaky	Test opened /admin before the login cookie was set	f2610e5	Selenium
B-17	20 Oct	Admin session list showed a blank model	modelId hard-coded to an empty string	0f3f3f7	Live run
O-01	open	Channel A-1 anomaly never detected	Absolute 0.05 error guard in the threshold	Planned: relative guard	Phase 2 analysis
O-02	open	Re-running a replay duplicates alerts	Replay clock restarts at the same time	Mitigated by /api/reset	Live run
O-03	open	Requests time out above about 55 users logging in at once	Database connection pool (15 connections) runs out while argon2 hashes passwords	Open (Jira OS-70)	JMeter NFT-02, NFT-03
B-18	22 Oct	CI backend job red on every push since 7 Oct	CI installed only backend/requirements.txt, so importing the pipeline failed on torch; fleet tests also needed the dataset	CI step installs CPU torch; fleet tests skip without data. Commit 8ad527f	GitHub Actions

T-01	22 Oct	JMeter read-only group returned 85% errors	Test plan cleared cookies each iteration, logging the once-only users out (test defect, not app)	Fixed in the .jmx	JMeter
T-02	22 Oct	Selenium IDE suite run stopped early	IDE 4 kept playing back in the tab left on /api/reset (tool issue)	Tests run one by one	Selenium IDE

The point worth making is that most of these were found by testing rather than by luck: B-11 (the blank globe) came from the browser console in a headless check, B-14 and B-16 from Selenium runs, B-13 and B-15 from looking closely at the redesigned screens, and T-01 from the JMeter run on 22 October, where an error rate of 85% on the read-only group pointed to the test plan rather than the application.

Soak test memory: the backend's working set was sampled every 30 s during NFT-04. It started at 241 MB and ended at 313 MB, with a peak of 342 MB over 20 samples. Most of the rise came in the first two minutes, when I started SMAP and Aqua replays for the screenshots in Chapter 9 and their LSTM models were loaded; from the fifth minute on it stayed between 306 and 313 MB. So the soak ran with live monitoring in the background, which is closer to real use, and shows no sign of a leak over ten minutes. Ten minutes is short for a soak test, and a run of several hours is still to do.

Performance in short: 50 users who log in on every loop were served with no errors, 50 read-only users averaged 85 ms, and a 10-minute soak showed steady memory and response times. The stress and spike runs found the limit: above about 55 users logging in at the same moment, requests time out because the database connection pool runs out while passwords are being hashed. This is logged as open item O-03; for the intended use, a few analysts who log in once a shift, it is well outside normal load.

9. Results and Discussion

9.1 Output screenshots of the working system

These were captured on 22 October 2026 at 1366 x 900 from the production build while SMAP and Aqua replays were running. The alert list mixes two kinds of rows: the eight seeded demo alerts (channel shown as 'unknown', raised 2026-09-23), which the Selenium tests depend on, and alerts raised by the live replay (real channel IDs such as C-2, A-5 and D-11, with the 2015 timestamps of the NASA data).

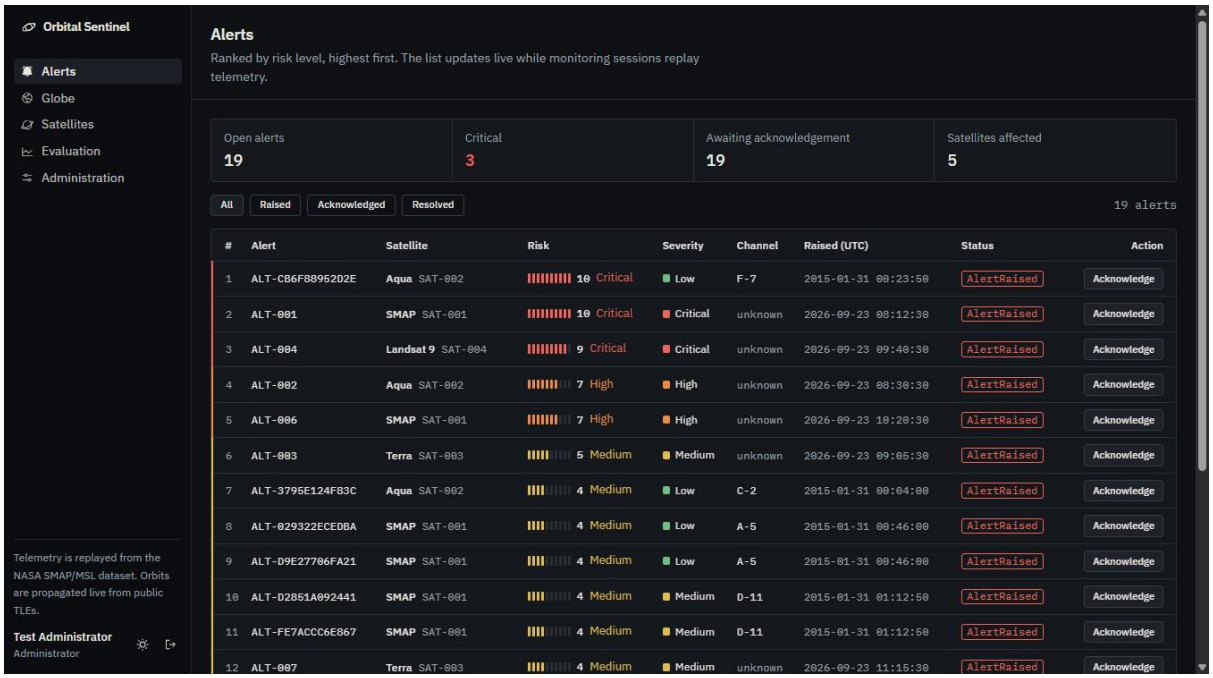


Figure 26: Alerts dashboard: seeded demo alerts and alerts from the live SMAP and Aqua replays, ranked by risk

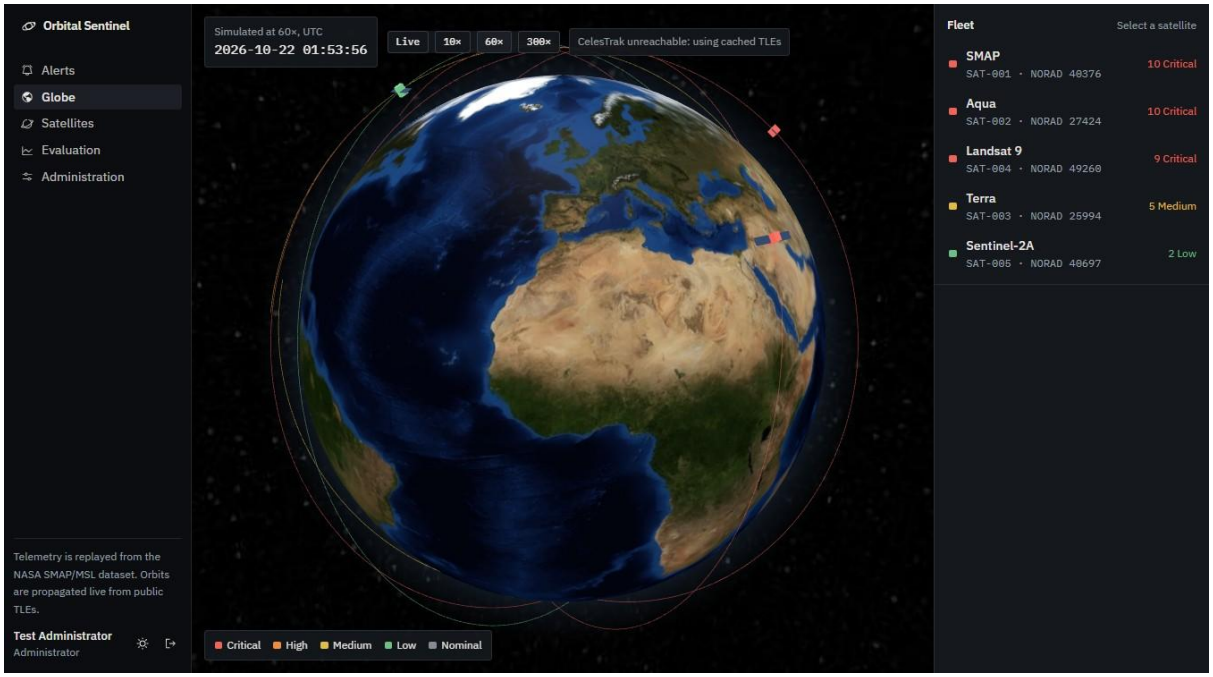


Figure 27: 3D globe with the five satellites, coloured by risk. CelesTrak could not be reached at the time, so the cached TLEs were used, as designed (NFT-11)



Figure 28: Health panel for SMAP: channel A-5 with the flagged range shaded. The latest detection shown is a seeded one, which is why its attribution reads 'unknown'

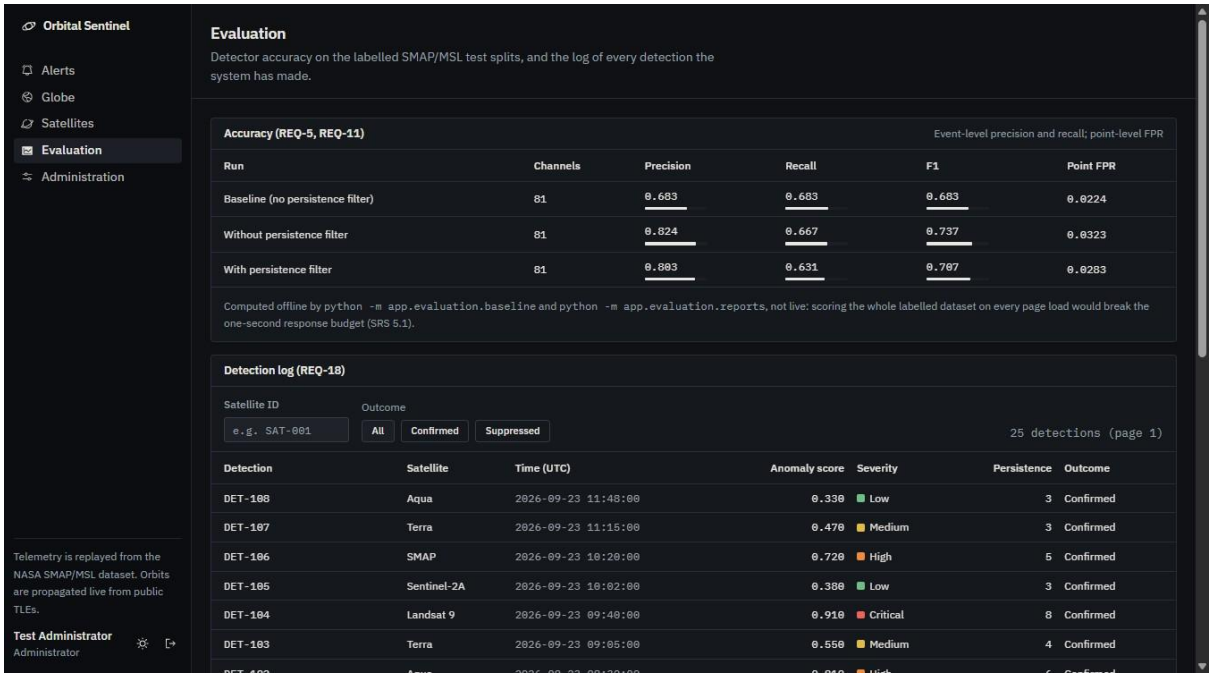


Figure 29: Evaluation page: accuracy with and without the persistence filter, and the detection log

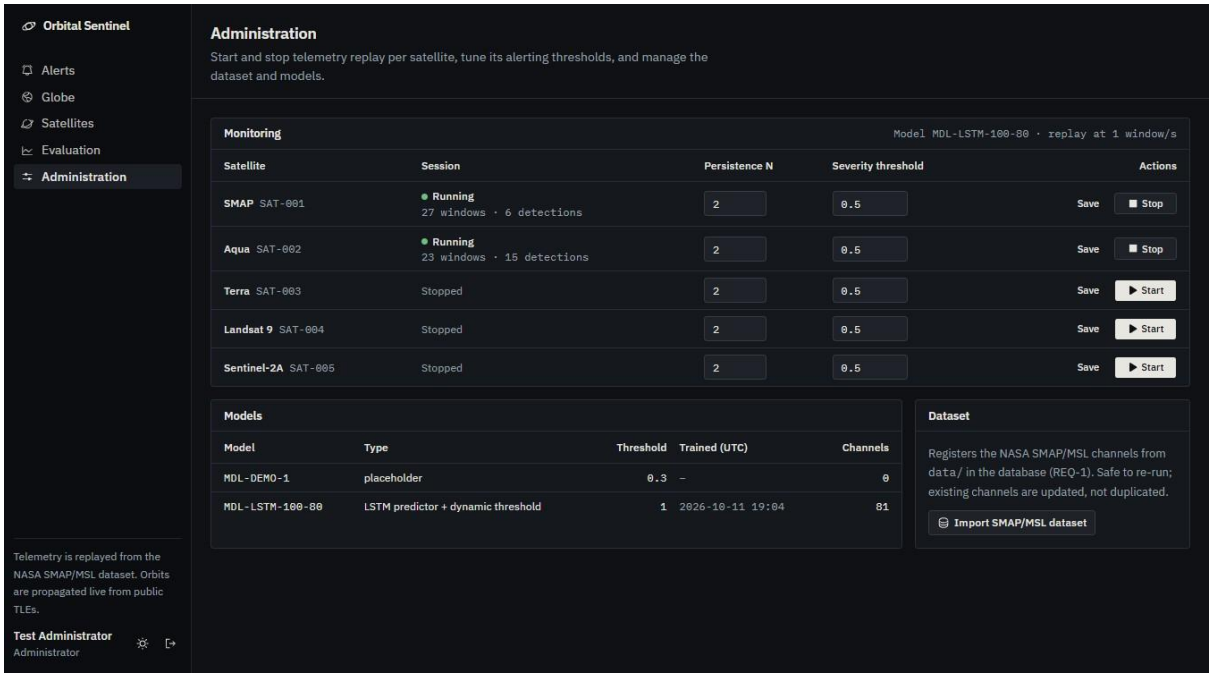


Figure 30: Administration: SMAP and Aqua sessions running, with per-satellite thresholds

9.2 Analysis against the objectives

Table 19: Results against objectives

Objective	Result	Status
1. Detect anomalies with a learned model and measure accuracy	All 81 labelled channels: precision 0.683, recall 0.683, F1 0.683, point FPR 2.24%. 71 of 104 labelled anomalies found, 33 false alarms. SMAP F1 0.711, MSL 0.630	Met; accuracy moderate
2. Explain, filter, score and rank	Attribution named the right channel for 15 of 17 labelled anomalies in the spike (random guessing would give about 25%). With N = 2, falsely flagged points fell from 3.23% to 2.83% on held-out data, recall fell from 0.667 to 0.631, F1 from 0.737 to 0.707, and false-alarm events did not fall (12 vs 13)	Met; filter gain small
3. Live globe, coloured by risk, click to inspect	Five real satellites from CelesTrak TLEs and SGP4; SMAP shows about 686 km altitude, 7.52 km/s, 98.3 min period; Selenium TC09 and TC15 pass	Met
4. Risk-ranked alerts with drill-down and a full log	Selenium TC04, TC05, TC08 and TC13 pass; every detection is logged with its outcome	Met
5. Roles and non-functional targets	403 for analysts on admin routes (pytest, TC07, TC10); scoring 632 to 838 ms per SMAP window; 50 users with 0% errors, but failures above about 55 users who log in repeatedly (O-03)	Partly met; FPS and Lighthouse pending

Two results deserve comment. First, the detector's F1 of 0.683 is lower than the 0.789 measured in the Phase 1 spike. That was expected and was recorded in ADR 001 before the full run: the spike picked its winner on the same held-out channels it reported on, so its number was optimistic. The original paper reports roughly 0.88 precision and 0.80 recall with a larger model and two extra steps (an inverted-error pass and horizon averaging) that this implementation leaves out.

Second, the persistence filter helped less than I expected. Many labelled anomalies in this dataset last only two or three 70-step windows, so any N above 2 starts to throw real anomalies away; at N = 3 the tuning half kept only 6 of 20. With N = 2 the filter mainly stops a single noisy window from raising an alert, which shows in the point false-positive rate but not in the

count of false-alarm events. Only 20 of the 104 labelled anomalies fall in the tuning halves, so N was chosen from a small sample; a dataset with longer anomalies, or a filter that counts time rather than windows, would be the next thing to try.

10. Conclusion and Future Enhancements

Orbital Sentinel shows that the published LSTM detector can be turned into something an analyst could use: every alert names the channel behind it, one-off spikes are held back, alerts carry a severity and a risk level, the fleet is ranked, and the satellites can be found on a live globe. The system was built in increments that each delivered a working demo, and every requirement in the SRS is implemented and traced to code and tests. Its accuracy is reported honestly on all labelled channels, including where it falls short.

The project also taught me some engineering lessons. Measuring on the data you tuned on gives numbers you cannot trust, and fixing that cost 0.1 of F1 but made the result defensible. Several of the worst defects, such as the blank globe and the database lock, only appeared when the whole system ran together, which is why end-to-end tests and live runs mattered as much as the unit tests.

Future enhancements:

- Replace the absolute error guard with one relative to each channel's own training error, so channels like A-1 can be flagged (fixes O-01).
- Keep the replay clock running across restarts so a second replay does not duplicate alerts (fixes O-02).
- Find and fix the backend hang under stress (O-03), then limit how many argon2 logins run at once so heavy login traffic slows down instead of failing.
- Add the inverted-error pass and horizon averaging from Telemanom to lift recall.
- Try a transformer-based detector and a persistence rule based on time rather than window count.
- Attribute anomalies jointly across channels on a spacecraft whose channels are genuinely coupled.
- Connect to a real operator feed, move to Postgres and support several users.
- Send e-mail or SMS notifications for critical alerts.
- Run inference on board (edge deployment) and send down only the explanations.

11. References

[1] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in Proc. 24th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD), London, U.K., 2018, pp. 387-395, doi: 10.1145/3219819.3219845.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[3] D. A. Vallado, P. Crawford, R. Hujsak, and T. S. Kelso, "Revisiting Spacetrack Report #3," in Proc. AIAA/AAS Astrodynamics Specialist Conf., Keystone, CO, USA, 2006, Paper AIAA 2006-6753.

[4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413-422.

[5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NIPS), 2017.

[6] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019.

[7] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998.

[8] T. S. Kelso, "CelesTrak: NORAD GP element sets." [Online]. Available: https://celestrak.org

[9] P. Fleith, "NASA anomaly detection dataset SMAP & MSL," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/patrickfleith/nasa-anomaly-detection-dataset-smap-msl

[10] satellite.js: SGP4/SDP4 in JavaScript. [Online]. Available: https://github.com/shashwatak/satellite-js

[11] V. Asturiano, "globe.gl." [Online]. Available: https://github.com/vasturiano/globe.gl

[12] FastAPI. [Online]. Available: https://fastapi.tiangolo.com ; Next.js. [Online]. Available: https://nextjs.org

[13] Selenium IDE. [Online]. Available: https://www.selenium.dev/selenium-ide/ ; Apache JMeter. [Online]. Available: https://jmeter.apache.org ; Atlassian Jira. [Online]. Available: https://www.atlassian.com/software/jira

[14] NASA, "Open MCT: Open Mission Control Technologies." [Online]. Available: https://nasa.github.io/openmct/

[15] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD), Anchorage, AK, USA, 2019, pp. 2828-2837, doi: 10.1145/3292500.3330672.

[16] N2YO.com, "Real time satellite tracking and predictions." [Online]. Available: https://www.n2yo.com

[17] Heavens-Above. [Online]. Available: https://www.heavens-above.com

Appendix A: Project Links

Item	Link	Notes
Source code	https://github.com/ItsPM25/Orbital_sentinel	README has setup steps; data and models are documented separately
Demo video	[Google Drive link to be added after recording]	Access set to 'Anyone with the link can view'
Jira project	https://vitstudent-team-twnkg2u9.atlassian.net/jira/software/projects/OS/boards/1	Viewer access can be given to the evaluator

Appendix B: How to run the project

```
# Backend (port 8000)
cd backend
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt -r ..\ml\requirements.txt
.venv\Scripts\python -m alembic upgrade head
.venv\Scripts\python -m app.seed
.venv\Scripts\python -m uvicorn app.main:app --port 8000

# Web (port 3001), production build
cd web
npm install
npm run build
npm run start

# Tests
cd backend; .venv\Scripts\python -m pytest -q
bash testing/run_jmeter.sh            # JMeter scenarios, HTML dashboards in testing/results/
```